Identity Precedes Staging: one play, many stages

Alvaro Rivera

2026-08-07

Abstract

A program's domain is usually written together with the code that deploys it, so a new place to run, or a new kind of client, means editing the domain. This paper reports one that was not. A game's rules — holding no input or output, naming no framework in its build graph — ran on five deployment configurations and were read by six clients, the last of them across three machines, at zero edits to the domain and zero test doubles in its own tests. A deployment is called a *staging* throughout, after theatre: one script, many productions.

The mechanisms that allow this are old and none is claimed. What is claimed is a property of the domain, measured against an orthodox ports-and-adapters version of the same game built for the comparison. That baseline matches the zero per staging. It *decouples* a domain where this arrangement *closes* one, on two independent measurements: the domain declares no obligation outward, where the baseline declares three driven ports — whence its stand-ins, its public surface, and the state it admits from outside — and its reconstitution needs no surface of its own, where the baseline grew one at 56 lines added and 5 removed. Calling those two *closure*, and reading the invariance as an *identity* prior to any staging, are readings and not further measurements. The paper is analytic in Gregor's (2006, Type I) sense, and offers a question to put to a system already built rather than a prescription. Two limits bound it: the domain is a pure computational core that causes no external effect, and its author commissioned the work.

Identity Precedes Staging: one play, many stages

TL;DR

A domain and its deployment are usually written together, so a new place to run means editing the domain. They can be separated completely, and this paper measures one that is. A Tetris board was held fixed while five stagings and six clients changed around it: zero domain edits, zero test doubles. The domain declares no interface for what drives it or for what it emits, so the contract that would be a port is provided by the substrate and bound in the staging. None of that geometry is new. What the paper adds is a measurement of it,

against a ports-and-adapters version of the same game built for the comparison — which matches the zero per staging. What it does not match is of another kind: ports and adapters *decouples* a domain, while this arrangement *closes* one. Two measurements: a ported domain declares obligations outward — three driven ports, and with them three stand-ins without which twenty of sixty-four tests do not run, an operation per port on its public surface, and a state it will accept from outside — and it needs a reconstitution surface of its own, costing 56 lines. This one declares none and needs none, replay re-entering by the operations the acts were performed through. Calling those two *closure* is a reading of them rather than a third count. Two findings arrived unsought, neither of them this paper's result and both bounding how it may be read: an incomplete record answers plausibly rather than failing loudly, so carry the recorded state and assert a replay against it; and a fact a domain emits must be derivable inside one actor, which bounds how a domain may be cut.

Dependencies. This paper is part of the Puppeteer Papers, a series of self-deposited preprints, and rests on four of them: the actor's speech and `tell` *(Paper 4),* `Reaction` *read as the recognition of a routine (Paper 3), the server read as an accidental category, ephemeral after bootstrap (Paper 7), and testimony (Paper 8), whose observer receives an account it is told. Paper 8 noted, without pursuing it, that a narration received is not yet a narrative recognized; this paper takes up the recognition, and reaches it not as its subject but as a consequence of the identity it argues. Methodologically it is an analytic theory contribution in the sense of Gregor's (2006) theory for analyzing (Type I): identity and staging are constructs by which an architecture may be described and compared, and the paper offers no prescription for building one — its instrument is a question that can be asked of a system already built (§10), not a rule for making one. The labs of Appendix A are accordingly an existence proof of realizability, not a design-science evaluation of cost or benefit, and where the text has slipped into evaluative language the slip is corrected rather than defended. Stated in the vocabulary of software-engineering research method rather than of information systems: the contribution is a descriptive model whose validation is by example — a system built and measured — in Shaw's (2003) classification of contribution and validation types. And in the terms of Stol and Fitzgerald's (2018) framework, it trades generalizability over actors and realism of context away entirely, being one domain on one framework, in exchange for a precise and reproducible measurement of a single quantity.*

1. The Domain and the Deployment

In most systems, the code that defines what a system does is not cleanly separable from the code that defines where it runs and which clients it serves. Moving a domain from a console to a web server means editing it; adding a second kind of client means adding to the thing the client observes; splitting one process into several is treated, uncontroversially, as building a different system. Where a system runs, and whom it serves, are treated as properties of the system itself

— so that a monolith and its distributed successor are counted as two systems rather than one system deployed two ways.

There is a field in which this separation is routine rather than aspirational: theatre. A play is written once and staged in many venues, for many audiences, without being rewritten, and no one counts the Broadway production and the school-gymnasium production as different plays. The term is taken from there: throughout this paper a *staging* is a deployment — a particular place a domain runs and a particular client it serves — and the domain is what is staged. Two units are counted in what follows and they are not the same: a **staging** is a deployment configuration, of which the example has five, and a **host** is an executable project, of which it has twelve, eleven reaching the domain through the actor and one writing bytes to a pipe and reaching nothing. Counts of hosts are counts of projects that had to compile and run; counts of stagings are counts of arrangements the domain was placed in. It is a naming convenience, not an argument, and earns its keep only if the separation it names can be made real and measured — which is the rest of the paper.

The claim is not that the domain is *decoupled* from its deployment, which is ordinary advice. It is that the domain is **closed**: it declares no obligation for a deployment to meet, so the dependency between the two runs one way only, and that direction is a checkable property of the built system. The domain compiles and runs with no staging bound to it. Every staging refers to the domain; the domain refers to no staging. In that precise sense the domain comes first: a staging is constructed against a domain that already exists and does not know of it. This is all the title means by *precedes* — a direction of dependence in the build, not a claim about time or metaphysics — and §2 measures it directly.

This is not dependency inversion under another name, though the difference takes some care to state and §9 states it. Ports-and-adapters architecture keeps adapters outside the domain, and for whatever a domain needs *from* the world it has the domain declare an interface and depend on it. In the model examined here the domain declares no interface at all — not for what drives it, nor for what it emits. It is entered by calls on its own operations, and it emits facts under logical names, while where those facts go, to whom, and what drives them are bound entirely outside it. Whether that is *cheaper* than the ported alternative is a separate question, and mostly it is not: a ported version of this game was built and counted, and a new transport costs its domain nothing either (§9, Appendix A). What the ported design still has, and this one does not, is an open obligation — an interface it declared and depends on someone to supply. That difference, not a cost, is what §9 measures.

Placed in the series, this is a question of a specific kind. Paper 4 asked who may speak; Paper 8 asked who may decide what an output is; the series puts its questions in that form. This paper asks a prior one — not who is entitled, but what an entitlement is attributed to, since a subject that does not hold still across its stagings cannot bear one at all. It is a question about the identity of a domain, not about infrastructure or topology. A later paper in the series will

need that question answered before it can treat a set of domains as a reusable repertoire; that is not this paper's concern, which is the narrower, checkable claim that a domain's identity is prior to and independent of its staging. One consequence is developed in §6 and §7: when a domain's acts are recorded as they are performed, the sequence of what it did is available whole from its journal — read from that record, wherever a copy of it is kept, rather than assembled out of fragments held separately.

One question follows, and a second that the first makes unavoidable. The first (§2, §5): does anything actually hold a domain fixed while its staging changes — across both where it runs and which client it serves — can that be measured rather than asserted, and what does the invariance reveal? The second (§3–§4, §6) is not a parallel contribution but what the first needs in order to be legible at all: since a client never reads a domain directly, through what does it read, and does it read the domain's present or its past? An invariance nobody can observe is not a result, so the reading has to be accounted for; that is the whole of the second question's standing here, and §6 says where it stops.

One thing about the order of the work belongs here too, because it decides how the rest is to be read. **The arrangement preceded its account; the experiments were built to expose and bound it.** The domain, the actor and most of the stagings of Experiment A existed before *identity*, *staging* and *closure* were words for anything here, and none of them was built to satisfy a construct. The labs are the other half: the ported baseline, the three containers, and the growth of §8.2 were made for this argument, and §8.3 says plainly where that weakens what they show. So the constructs are instruments for reading a construction rather than a design derived from a theory, and the question they are answerable to is whether they describe faithfully something already built — which is the sense in which the labs are validation by example and not a test of a hypothesis.

2. Two Experiments

The claim of §1 is checkable, and this section checks it against a running example: a Tetris game, whose board and rules are a small but complete domain, written as a `Well` and a set of operations over it. The domain is held fixed, and two things are varied around it in turn — where it runs, and which client it serves. The measurement in each case is the same: the number of changes the variation forces on the domain source. The evidence is the git history of the example repository, and the anchors are collected in Appendix A.

The domain's independence has a concrete form before anything runs — the shape of the build. Every executable host in the example reaches the domain through a single actor, and the domain reaches back to nothing but the language's own library:

```
11 hosts           -> TetrisActor
   TetrisAi, TetrisConsole, TetrisObserver, TetrisServer, TetrisStage,
```

```
    TetrisStageDuo, TetrisStageDuoTls, TetrisStageServer, TetrisWatch,
    TetrisWeb, TetrisWebRest

TetrisActor        -> TetrisDomain, Puppeteer, Choreography
TetrisDomain.Tests -> TetrisDomain

TetrisDomain       -> (none)
```

Figure 1. Every declared dependency between the example's projects, read out of the project files on its main branch rather than drawn, and grouped so that the direction is visible: an arrow means *this project is built against that one*. Every staging depends on the same domain; the domain depends on none. Fourteen of the fifteen projects appear — `TetrisSend`, the pipe writer, is omitted because it references nothing and is referenced by nothing, being a separate executable that only writes bytes. Read downward, every arrow leads toward `TetrisDomain` and none leads away from it.

The figure answers one question, *who depends on whom*, and it is worth saying what it does not answer. It does not show that the domain leaves nothing open — no port declared, no contract, no persistence or transport requested. That is a different property, it is the stronger of the two, and it is measured separately against a built alternative in §9. A reader unfamiliar with this language's build files needs only the arrow: a project that names another cannot be compiled without it, and a project that names none can be compiled alone.

Declaring no reference is not the same as naming nothing, and one qualification belongs with the figure rather than after it. Three hosts do name a domain type, reaching it through the actor's reference, which in this language is transitive: each writes `typeof(TetrisDomain).Assembly` to hand the assembly to the framework (`sm-duo-tls/Program.cs:33`, and the same line in `sm-server` and `sm-duo`). What they name is an empty public anchor and nothing else, so a staging holds a name and not a handle. Lab E reads that fence at the level of members and reports its two authored exceptions.

What the domain does instead is *satisfy conventions*, which is a different relation from declaring a dependency and the difference is load-bearing. The framework finds the domain by reflection, so its operations must be ordinary methods with coercible parameters, and the empty public anchor is there so a host can name the assembly in its own source — a convenience, not a requirement, as Lab F reports. What separates conformance from dependency is not interpretive: the domain compiles and its own tests run with the framework absent from the build graph entirely (Lab E). A dependency cannot be absent from the build and still be met; a convention can.

That direction is what §1 called *precede*, and one note on vocabulary avoids confusion later. With arrows running from a dependent to what it depends on, the domain is a *sink*: every staging's references lead to it and none leads out. Reverse the arrows and it is the *root*, every staging reachable from it and

none from another. Where later sections call the domain the stagings' common ancestor, that reversed orientation is what is meant; §5 says what the property does and does not establish.

Experiment A — change the stage. The same `Well` is run on a console; in a browser, with input and output carried over a real WebSocket connection, and in a second browser variant over HTTP with server-sent events; across two co-hosted actors coordinated by a StageManager, once in memory and once over a real TLS channel; and across three separate Docker containers, each a peer joined to the others over real container-to-container TLS. These are genuinely different deployments — different processes, machines, transports, and wire formats. The domain source is identical across all of them: a diff of the domain directory between the first staging and the last comes back empty (Appendix A). What changes from one staging to the next is host code — the shell that binds an input source and an output sink to the actor. The example's own name for that shell is exact: the host is "an accidental shell" (`actor/IGameHost.cs:14`), and the same `Well` runs on `PerformanceV2` or on `StageV2` without knowing which. The example builds clean and its domain tests pass, unchanged, in every staging.

A distributed demonstration invites more credit than it earns, so its boundaries belong in plain sight. Three containers run the same `Well` as three StageManager peers over genuine container-to-container TLS; a scripted driver on the Director plays the game, and the two casts converge to a byte-identical board. Adding that staging left the diff of both the domain and the actor directories empty. What is *not* claimed is any test of resilience: no peer was killed and no partition induced, so partial failure is untouched (§9). The setup, the bootstrap, and what each node is handed are in Lab B.

Experiment B — change the client. With the stage held at one host, the client is varied instead. The same board is played by a human at a keyboard reading an on-screen grid; by an automated player that sends moves over a pipe and reads the board through a view it computes for itself; by two passive observers, one that pulls the board and one that receives it pushed; and by a browser, in which a player and a spectator are both written in JavaScript and reach the game over the network. Six clients over one `Well` — some in C#, one a PowerShell script, two in a browser — and again the domain directory does not change across any of them (Appendix A). Six is the number of additions; counted by how differently the board is actually represented, the labs hold two, a grid and a vector of column heights, and Appendix A reports both numbers rather than the flattering one alone. What each client adds is an input adapter, an output adapter, or both; none of them is a change to the game.

The two experiments vary independent things. Experiment A changes where the domain runs; Experiment B changes who observes it and how. Neither touches the domain, and that independence is the result: its constancy does not depend on where it is staged or on who is watching. The identity §1 argued to be prior to the staging appears here as an invariant the two variations cannot move — and the build graph says why it cannot be moved: nothing the domain refers to

changed, because it refers to no staging and to no framework in the first place.

How much these counts separate this arrangement from a competent ports-and-adapters one is a question §9 answers with a baseline that was built and counted, and the answer is less than the counts alone suggest: a hexagon's rule model takes a new transport at no cost either. What the counts establish here is the invariance itself, not a saving over an alternative.

3. What a Client Sees

Experiment B added clients by adding adapters, and the word carries the second half of the argument, so it is worth making precise. A client never reads the domain directly. It reads through an adapter, and the domain is the same behind every one of them.

What the domain emits is not a rendering — but calling it a *raw* fact would overstate the case, and the difference needs a criterion rather than an adjective. What is emitted is the set of occupied interior cells, and that set is computed rather than stored: the domain unions the settled pile's cells with the falling piece's and clips the result to the frame's interior (`domain/Well.cs:322-331`). The pile itself holds a set of positions (`domain/Pile.cs:26`). So what leaves the domain is a derivation the domain performs, not a field read off storage — and the domain project, as §2 established, declares no dependency on the framework or on any staging: it has no notion of a screen, a renderer, or a client. The substrate pushes that set outward on each move (`actor/TetrisActor.cs:43-47`), forwarding it rather than projecting it further.

Derivation, then, cannot be the line, since the automated player's column profile is a derivation too. The criterion this paper uses is the one the series applies elsewhere, and it is a single question a reader can put to any domain's source: **does a rule of the domain depend on the notion?** If some operation is stated in those terms, the notion is the domain's own, and what is emitted in them is a fact it holds; if no operation mentions it and only a consumer needs it, it is a view computed for a reader. Applied here the two separate cleanly. Occupancy by position is the vocabulary the rules are written in — collision is a membership test over cells, and what counts as a complete row is counted in them — so the emitted set is in the domain's own terms. Column heights, the gaps between them, their bumpiness appear in no operation of the `Well`; nothing in the game is decided by any of them, and only the automated player wants them. That is why the one is emitted and the other computed outside.

What the criterion buys is that the line is inspectable rather than asserted: a third party can read the domain and check which notions its rules turn on. What it does not buy is a line that is sharp or permanent. Which notions a domain owns is a modelling judgment — Paper 8 called this the negotiable boundary of the two it drew — and a `Well` that grew an operation deciding something by column height would move it, legitimately. The claim here is not that the

boundary is objective, only that it is auditable, and that in this domain the audit comes out as this section describes.

Each client turns the fact into something it can use, and the turning is the client's work, not the domain's. The human's on-screen grid is produced by a renderer that lays the bitmap out as squares (`actor/BoardRenderer.cs:19-48`). The browser produces the same grid a second time, independently, in JavaScript — its code is commented as mirroring that renderer (`web/Program.cs:169-178`) — so the terminal grid and the browser grid are two adapters over one frame. The automated player needs something else: it reasons poorly over a bitmap, so it lifts the same frame into a column-height profile — a skyline, the gaps and wells between columns, a few aggregate figures (`tools/pile-scan.ps1:54-83`) — a form suited to deciding where a piece should go. None of these is the board's own appearance; nothing in the domain decides anything by a picture. It emits a set of cells in its own terms, and each client projects that through an adapter chosen by the host that runs it.

One fact about how that adapter came to exist belongs in the argument rather than only in the disclosure at the end of this paper, because it is the sharpest evidence available for the claim of this section. The automated player is an instance of a large language model, and the column-height view it reads the board through was written *by that instance* during the lab, with the author's permission, to suit the form in which it reasons (`tools/pile-scan.ps1`). Nobody translated the board for it. It was handed the same emitted fact every other client gets, and it authored its own projection of that fact into the shape it could use. A projection belonging to the party that reads it is, in this one case, not an interpretation of the architecture but an observed event: the reader was a different agent from the domain's author, and it built the adapter itself. How that came about, and what it means for weighing the evidence, is disclosed in the acknowledgments.

This is where the on-screen grid loses its privilege. It is natural to treat the pixels a human sees as the real board and the automated player's vector as a derived abstraction of it. The system does not: both are adapters over the same emitted fact, and neither is closer to the domain than the other. The grid feels primary only because it is the one a person happens to read; the domain has no such preference, because it produces neither — it produces the fact both are made from. The automated player makes this hard to miss precisely because its adapter looks nothing like a human's. Faced with a client that reads the board as a vector of column heights, one notices that a human, too, only ever reads it through a rendering, and had simply stopped noticing.

This arrangement has a much older statement in the database literature, and the paper is better for saying so. Logical data independence — that an application should be insulated from the structure in which data is held, and served instead by views derived from it — was part of the relational model from the beginning (Codd, 1970), and the three-schema architectures that followed made the separation of a stored schema from per-application views explicit. "The do-

main emits facts and each client projects them" is that separation, reached from the side of the domain. What differs is who owns the projection. Under data independence the mediating system owns the schema and derives the views: the projection is the database's work, performed on the application's behalf. Here there is no mediating schema authority at all — the domain emits its own facts in its own vocabulary, and each client's projection is authored outside both the domain and any central schema, by the staging that binds it. The insulation is the same; the party doing the insulating is not.

This connects directly to the boundary Paper 8 drew. There, what an output *is* — its projection — was shown to belong to the actor that authors it, not to the domain that holds the material; a domain object that renders itself reaches past what it knows into a decision it does not own. Here the same boundary is seen from the client's side: the projection is authored per client, outside the domain, and the domain that would have rendered itself instead emits a fact and lets each client author its own view. The adapters are the input sources and output sinks of the earlier papers — a keyboard or a pipe on the input side; a terminal grid, a numeric vector, or an HTML canvas on the output side — bound to the actor from outside and exchanged without the domain's knowledge. Whether a client is served by pulling or by being pushed to is a choice of the same kind, made in the same place, and the two modes have long been catalogued in the publish/subscribe literature (Eugster, Felber, Guerraoui, & Kermarrec, 2003). That the domain does not know which adapter is attached is the same fact, seen once more, that let the stage and the client vary in §2 without moving it.

4. Actor and Audience

Section 3 answered *through what* a client reads the domain: an adapter, never the raw domain, and no adapter privileged. A second question sits beside it — what, in time, a client reads — and the answer is narrower than the temporal reading of it invites. Acting and watching are distinct roles and what each may have differs; that is a separation of roles, not a law about time. The distinction matters, because the temporal version of the claim is either trivial or false.

Take the obvious shortcut first, since it is the one that has to be dismissed with care rather than in passing. Each mutating command could return the board it produced, and the client that issued the move would have its picture at once. There is nothing incoherent in that. It is *read-your-writes*, one of the named session guarantees (Terry et al., 1994); it is usually what a commanding client wants; and it is often chosen for good reasons of latency. What such a response cannot do is serve a client that issued no command, and that limit is close to definitional — no request, no response to be served from. So what follows is a design corollary rather than a logical necessity: a system whose only path to a view is the command's response has no path at all for an observer that does not command, and every client in §2 and §3 that was not driving the game is exactly such an observer.

This is not a new mechanism, and — the point deserves to be conceded rather than discovered — the temporal reading of it is not new either. The field already separates the two operations: command-query separation, and command-query responsibility segregation after it, hold that a view is not served from the write (Meyer, 1988; Young, 2010), and event sourcing builds a view as a projection over recorded acts rather than a snapshot of current state (Fowler, 2005). And that what an external consumer holds is the past was argued two decades ago in the vocabulary of data: data outside a service lies beyond the reach of that service's transactions and is *temporally disconnected* from it, so what has left is what was and not what is (Helland, 2005). Nor is the structure beneath that new: in distributed systems it is the causal past. Lamport (1978) showed that the events available to an observer form a partial order — what can have reached it is what happened before it — so an observer's knowledge is constitutively of the past, and a "present" shared across separated parties is not a thing that can be observed at all. This section therefore does not claim the past-tense view as a finding; it sits inside a structure distributed systems have had since 1978, and the data literature stated for consumers two decades ago.

Nor is the present unavailable to an audience, and this paper's own labs say so: one of the six clients polls the board and is handed the current state (Lab C). An audience can see the present. What differs by role is narrower — a live read taken *in order to act* belongs to the party acting. The game's host queries the current board, is a piece falling, so as to choose its next move, and that reading is part of the act; an observer polling the same board is not deciding anything, and is being shown a configuration. The difference is one of standing, not of timing, and in that form the claim is not this paper's to make: Paper 4 gave the actor its voice, and Paper 8 established who is entitled to say what an output is. This section applies their result to the seats in a theatre rather than extending it.

Naming the two apart is where this paper needs the one before it. An observer given only the present is given a snapshot — the configuration as it stands — from which, to learn how that configuration arose, it must reconstruct the history; that reconstruction is the defect Paper 8 traced. An observer given the past is given the account itself, the sequence of acts the domain performed, which Paper 8 named *testimony*: knowledge held on the word of the one that lived it, not rebuilt from stills. The record-reading view of this paper is therefore not merely consistent with Paper 8 — it is Paper 8's resolution made operational. To see a domain by reading its record is to receive its narrative as testimony, and it is the only way an audience comes to know *how* the state arose. What polling supplies is the configuration; what the record supplies is the account.

This returns to what the paper is about. Because what an audience is *told* is the recorded narrative rather than a state read off the moment, the thing told is a standalone object: it exists in the record, the same for every audience, independent of who reads it and when. And it is the same for every audience for the same reason the board was — one domain, referenced by every staging,

produced it. The invariant §5 names — the domain as the common ancestor of its stagings — is what makes the narrative one narrative, readable the same from every seat. What remains the same when the staging changes is, in the end, what there was to see.

5. Identity

The two experiments leave a single structural property to state, and it is statable without interpretation: a domain was run on five stagings and read by six clients — between them, in two genuinely different representations — and through all of it the domain referred to no staging, every staging referred to it, and its content did not change. In the dependency graph the domain is a sink; in the reversed graph it is the root from which every staging is reachable, which is what §2 meant in calling it their common ancestor.

What that property licenses one to say is a second question, and the word this paper uses for it invites more metaphysics than the evidence supports. *Identity* is offered as a reading of the property above and not as a further finding: a thing that holds still under that much variation is the sort of thing an entitlement can be attributed to, which is what the rest of the series needs of it.

Two cautions follow, and they mark exactly where measurement stops. The first concerns the word *position*. A position in a graph individuates nothing — a different domain, or this one rewritten, would occupy the same one — so the property measured here cannot be what makes this domain the domain it is; nor is it even the graph's only sink, since the language's own library is another. The second is that the property is evidence about a *relation* and not a criterion of identity: it says that what the stagings were built against is one thing, and that its content owes them nothing. Calling that an identity is a reading of the evidence rather than a part of it, and the two are better kept apart than run together, because the measurement is checkable and the reading is not. What this paper measures is independence and priority. *Identity* is the word for what independence and priority license one to speak of, and the title should be read in that light.

One echo in that title is worth disowning before a reader supplies it. *Identity precedes staging* has the shape of Sartre's *existence precedes essence*, and inverts it — an essence prior to any concrete appearance is close to what that formula was written to deny. The resemblance is accidental and the claim is not the philosophical one. Nothing here is an essence: what precedes a staging is a position in a dependency graph and a source directory that did not change, both of which can be read off a built artifact by someone who has never heard of the phrase. A reader who hears the echo should hear it as coincidence of form, not as an argument being smuggled.

Read that way, *precede* is not a claim about time or ontology but the direction of the dependence. A staging is built against a domain that already exists and does not know of it — the way a library is compiled before, and in ignorance of, the

programs that will call it. The domain does not *survive* its stagings; surviving would make the stagings the primary thing and grant the domain the modest virtue of enduring them. The relation runs the other way: the stagings were built because the domain was there to be built against. Nothing the domain refers to changed across the experiments because it refers to no staging at all — so its content was never at risk from a change downstream of it.

This has a second face, less formal than the graph and worth stating because it is what makes the identity legible to a person rather than only to a build tool. A domain with an identity is one whose parts can be understood as stable roles. To know what the board of the game *is* — that it holds a pile of settled cells, spawns and moves a falling piece, clears full rows, ends when a new piece cannot enter — is to know, structurally, what it does and does not do, what may be asked of it, and what it will never answer. This is not a psychological claim about how people understand stories; it is the observation that a role with a fixed identity fixes its own boundary, and that boundary is what domain-driven design reached for with its aggregates and bounded contexts — arrived at here by asking what stays the same, rather than by drawing a diagram of layers. The invariance measured in §2 and the legibility described here are the same thing seen twice: what holds still across every staging is what there was to understand in the first place.

6. Recognition

If a domain keeps its identity across every staging, then whatever it did keeps its identity too. The sequence of a domain's own acts — a piece spawned, moved, dropped; a row cleared — is recorded in its journal as the domain performed them, and because the domain is the same on every stage, that record reads the same from every one. This is the first, plain consequence of §5: the account of what happened is available directly from the record the acts were written to, whole wherever that record is kept, and does not have to be reassembled from fragments held elsewhere.

That availability is what lets a client do more than watch. Given the sequence of acts, a reader can recognize in it the routine the acts compose — that these three moves and a drop were the placement of one piece, that this run of placements filled a row. The term needs pinning down, since the rest of the section leans on it, and it is not this paper's to define: a *routine*, in the sense Paper 3 gave it, is a pattern over journal entries together with the correlation that binds them into one trajectory — a reaction seeks an opening entry and then a closing one, and what it matches, between them, is the routine. So recognition is an operation with a definition and an implementation, not a figure of speech, and it reads the record as it stands rather than inferring a history from snapshots.

Recognition in this paper is that operation applied to a domain whose identity holds: because the acts are the same across stagings, the routine they compose is recognizable across stagings too. That step has been measured (Appendix A,

Lab H). One reaction, seeking a spawn and then the drop that ends it, recognizes the same two placements in the same order over three different stagings: the journal of a single process, the journal a warm server keeps while driven over a named pipe, and the journals of the three containers — read inside each node, against its own store. Within that third staging the three nodes' records are byte-identical. *Across* the three stagings what matches is the sequence of acts — same count, same order, same shape, verb for verb — while the entry identifiers differ by a constant, the cluster writing three idempotent seeding acts where one process writes one. The domain's diff for the whole exercise is empty. So the claim is a result rather than an inference, on a short trajectory: two placements, not two hundred.

That pair of results is worth reading together, because what moved and what did not are not the same kind of thing. The identifiers moved: the record's bookkeeping is a fact about where the domain ran. The acts did not: a spawn on a console and a spawn inside a container are one operation with the same parameters, and each leaves behind an entry of the same kind. So what held still across the three stagings was not the record's representation, which shifted, but what the act was and what performing it left behind. A staging changes who is present when a domain acts and changes nothing about what the acting does. That is the sense in which §5's identity reaches past the source text — the invariance is not only in what the domain is written as but in what its verbs accomplish when performed — and it is Paper 4's account of an actor's speech that makes this a claim rather than another way of stating the empty diff.

Three things the lab found are more interesting than the confirmation, and §6 is narrower for them. The first is that this record offers no handle to correlate on: a spawn names a piece *type* and a drop names nothing at all, so the trajectory between them is tied by order alone and not by any identity the acts carry. The second is worse, and it answers part of the question this section leaves standing — whether a reading can be wrong. It can, and silently. Landing is not an act here: the board lands a piece inside a tick or a drop, privately, so a piece that comes to rest under gravity leaves its opening unclosed, and the *next* piece's drop closes it. The count comes out right and the correlation comes out wrong, with nothing to signal the difference. The third is that entry identifiers are not invariant across stagings — the three-node staging writes three seeding acts where one process writes one, so everything after shifts — and the closing identifier is the only handle a match hands its reader. The routine recognized is the same; the name by which a reader could refer to *this instance* of it is not.

The second of those generalizes beyond this lab, and it is the same shape as the finding of Lab G. A routine is recognizable only in the moments of it that were *acts*. Where a moment was a private consequence — a landing folded into a tick, a pile absorbing cells — the record holds no entry, and no pattern can seek what was never recorded. Recognition is bounded not by the reader's ingenuity but by what the domain performed out loud.

That bound exposes a gap in this paper, and it is better named here than left

for a reader to find. Section 3 offered a criterion for the *vocabulary* of a record — does a rule of the domain depend on the notion? — which is auditable against the domain's own operations. There is no companion criterion for its *granularity*: nothing in this paper says which moments must be acts. Yet granularity is what recognition rests on, so the fidelity of a record as an *account* depends on a decision the paper leaves unguided while the fidelity of its *vocabulary* has a test. And the failure mode is the worst kind, the one Lab H found: an absent act yields not an error but a smaller coherent story, matched to the wrong closing entry, with the count coming out right.

No criterion is offered, and the honest statement is that this paper does not have one. What its labs supply is three cases any candidate would have to accommodate, reported as three cases and not as a rule. Absorbing a landed piece had to become an act when the board was cut in two, because two parties had to speak and the pile's own record needed an act of its own (§8). A landing had to be an act for a reader that *reads* the record, since Lab H's reaction could not seek what was never written. And a score and a difficulty level did not have to be acts at all, being folds over acts already recorded (Lab I). Together they point at what a *reader* of a record needs to identify, as against what a *re-performer* of it needs — but three cases are not a criterion, and offering the hint as one would be the overreach §5 declines elsewhere.

It is worth locating the gap, since it is not peculiar to this arrangement. Choosing which moments a model records is a modelling decision of the same kind as drawing an aggregate's boundary, for which the domain-driven design literature offers heuristics and worked judgment rather than a test (Evans, 2003). What this arrangement changes is only when the mistake surfaces: a boundary drawn badly resists the code being written against it, whereas a moment that should have been an act surfaces much later, in a reader that recognizes the wrong thing and reports it to no one. That is an open problem this paper raises and does not close, and it is worth being exact about what kind of question it is. Which moments must be acts is not a question about representation — about how faithfully a record renders what happened — but about what the domain is *doing* when it performs one. This paper has no account of that.

One consequence of it belongs beside §8.2 rather than here alone, because it is the same cost seen from the other end. Growth in this arrangement is additive when the new thing is a value or a rule over acts already recorded — that is what Lab I measured. Discovering that a *moment* should have been an act is the other case, and it is the expensive one, because every record already written lacks that act and no projection over those records can supply it. What the domain would have to do is not add an operation but reinterpret a history that does not contain the distinction, which is exactly the schema-evolution cost §8.2 concedes to the event-sourcing literature (Overeem, Spoor, & Jansen, 2017; Overeem, Spoor, Jansen, & Brinkkemper, 2021). So the missing criterion is not only an intellectual gap: it is the decision whose mistakes are dearest to correct, and that is the strongest reason to want one.

This is where the paper meets the one before it. Paper 8 followed an output to the party that receives it and showed that, within a trust boundary, the receiver comes to know the state as *testimony* — an account it is told rather than one it reconstructs. But an account received is not yet a routine recognized: being told the sequence of acts is prior to seeing, in that sequence, what was done. This paper takes the next step, and reaches it as a consequence rather than a new claim — an account whose subject keeps its identity can be recognized as the same routine wherever it is told.

And the recognition is the client's, performed through the same adapter §3 described. The board does not announce "a row was cleared"; it records the acts, and each client recognizes the routine through the view it holds. The automated player reads its column-height vector and recognizes a well being filled; a person reads the grid and recognizes the same thing; neither recognition is the domain's, and neither is privileged over the other. What both recognize is one routine, because beneath both adapters is one domain with one identity — which is where the paper began. That no recognition is privileged does not settle whether recognition is itself governed: who is entitled to read a routine, and whether a reading of one can be wrong, is a question this paper reaches and leaves standing.

7. Where the Narrative Lives

Section 6's availability has a counterpart, and it wants naming carefully. When what a system does is spread across autonomous services, each with its own store, the sequence of what happened is in none of them. It has to be assembled afterwards, by correlating records that were produced separately — which is part of what distributed tracing and correlation identifiers are for. That work has a developed literature: Google's Dapper (Sigelman et al., 2010, a technical report rather than a refereed paper), and the refereed line that followed it (Mace, Roelke, & Fonseca, 2015; Sambasivan et al., 2016; Kaldor et al., 2017). None of that is treated here as a deficiency, and nothing in this paper says a system should do without it. The point concerns *completeness at a reader*: whether some single record holds the whole account, or whether the whole exists nowhere and must be joined together out of fragments.

Put that way, the section survives its own Lab B, which would otherwise contradict it — three machines keep three journals, converging by replication and by a catch-up path, so the narrative is in three places and reading it does depend on a mechanism. What makes that unlike correlation is not that there is one copy but that the three are copies *of one record*: each holds the same acts, each alone holds the whole game, and a reader at any node joins nothing to anything. So the honest form of the claim is *one logical narrative, replicated, with convergence the framework guarantees* — weaker than "one place", and what the labs support. It is scoped, too, to one actor's own first-person account (Paper 4): a system of many actors has many accounts, and relating them is a question this paper does not take up (§8). The model does not remove the need to know

what a system did; it changes where that knowledge already sits.

8. Limits, and Two Observations Nobody Was Looking For

The claim is bounded, and the boundaries are worth drawing, the more so because the result is easy to overstate into a promise it does not make.

8.1 What the claim covers

It holds only where there is something for it to hold of. A domain with a genuine identity, staged more than one way, is where the invariance has teeth; a program too small to have a domain distinct from its deployment, or one that will only ever run one way, has no second staging to be invariant across, and the apparatus is overhead. The claim is not that every program's domain is separable from its deployment in practice, but that where the two are genuinely distinct, the separation is real, buildable, and measurable — and that collapsing them, where they were distinct, is what the ordinary practice does.

A second limit, and the more consequential one, follows from what the demonstrated domain is. The `Well` is a pure computational core: it holds rules and state, and its outputs are data. It keeps no external store, provokes no effect in the world, reads no clock, authorizes nothing, and depends on no other party — it never has to *cause* anything. That is exactly the case in which "the domain emits facts and each client projects them" suffices, because every consumer really is a client: it wants to know, and nothing turns on whether it ever does.

What the example does not exercise is a domain that must bring about an external effect and then reason about the result — charge a card, notify a partner, settle a payment. There the consumer is not a client but an obligatory collaborator, and "the domain emits facts and each client projects them" is not a description of that relation. Nothing here was built that way, so nothing here measures it. Paper 4 (Rivera, 2026d) takes up cross-actor continuity and its `tell`, which is where a reader with such a domain would look; whether it settles this case is not a question these labs asked.

8.2 Invariance under staging, not under evolution

Identity here is invariant under staging, not under evolution. The claim is that a *change of staging* does not change the domain — not that the domain is frozen. Requirements grow, boundaries are redrawn, and verbs are retired; a domain that does any of those is still a domain no staging can call for a change to. Two measurements bound that, one in each direction.

The domain grew. A score and a difficulty level were added to the board — the authority case, a value any client could compute and which therefore has to be single-valued, and a rule over acts already recorded (Appendix A, Lab I). Nothing was deleted but doc comments, no signature changed and no verb removed, shown by diffing a sorted inventory of the board's members. What

the lab supplies that this paper had not measured is the converse of §2: when the domain grew, all twelve host projects kept running with no edit, checked by running them rather than compiling them. The cost of *adopting* the new capability then fell only where it was wanted — nothing for four of them, one line for five, four lines for each of the two browser hosts, and thirty-two for the input host, the one place where the level actually changes what happens.

The domain was re-cut. One role was modelled too large and became two, by authoring the two and then reading the original's recorded acts — the same read rehydration performs — and driving each new role to perform its own, so that each ends holding a record in its own voice (Lab G). The original's journal is not cut or rewritten; it is read as the account of what happened, and kept. Eleven of the twelve host projects were untouched while the domain divided beneath them, and the twelfth needed one line.

Where such a boundary may be cut has a criterion, and §8.4 relies on it: **a boundary is admissible where no decision on either side reads state the other party mutates concurrently.** The cut of Lab G passes it because the board's phases are disjoint — the pile is immutable while a piece is falling, and the piece is gone once it has been absorbed — so neither role ever reads the other's state mid-change. The criterion is about concurrency and not about size, and it is what makes a tight-looking split sound.

That second one could look like a threat: if the boundaries *within* a domain can be redrawn, is the identity being measured merely that of the current cut? The distinction the paper turns on answers it. A re-decomposition is a change to the domain, made by whoever authors it for reasons of modelling; it is not a change of staging, and no staging can call for one. What carries across is the record — the acts were performed and remain performed — so the append-only character of every record involved survives the redrawing untouched.

The shape of growth is additive, and one measurement supports that rather than establishing it. An operation added is a new act in the record rather than a reshaping of the acts already there, so growth accretes; Lab I is that shape observed once, on the authority half of the change. The half that adds an *act* rather than a value — a piece held and swapped, a move undone, a second player — is unclimbed here, and so is the retirement of a verb, which in a production system is marked obsolete and kept for as long as any recorded act still invokes it. Both are named as future work rather than claimed. Nor should additive growth be read as a claim that evolving a recorded history is free: empirical study of event-sourced systems finds converting recorded events as their schemas change to be a real and recurring cost (Overeem, Spoor, & Jansen, 2017; Overeem, Spoor, Jansen, & Brinkkemper, 2021). Nothing measured here bears on it. The invariance reported is across stagings of a domain, not across revisions of it.

8.3 Threats to validity

One threat to the result's validity cannot be argued away, and it belongs in the open. Every staging and every client counted here was written by the same author as the domain, and a metric of the form *the domain did not change* is gameable by construction: any code can be held constant if all variation is pushed outside it, and an empty diff is equally consistent with a domain designed backwards from a known set of stagings. What would discriminate between those is a staging nobody anticipated, introduced by a party with no part in the design.

The record supplies some of that and not all of it, and the part it supplies is checkable in the example's own history rather than asserted here. The domain's last commit is dated 29 June 2026 and it has not been modified since. Three stagings and clients were added on 23 July, twenty-four days later — a gesture client, a constrained-device client, and the three-machine deployment — in working sessions separate from the one that authored the domain, and the two clients appear in no earlier artifact. The domain's diff across all three is empty. Two of the three are not labs of Appendix A, having been withdrawn there for want of measured evidence about the clients themselves; what is cited here is only their effect on the domain, which was measured, and which is all this particular argument needs.

It remains weaker than an independent test. The author commissioned the work, and cross-machine deployment had been named as a next step in a host comment the day after the freeze. One structural fact bounds part of the worry without dispelling it: the fence is kept by the compiler and not by anyone's discipline, since the domain declares no dependency and its verb-bearing types are internal (§2), so staging concerns could not have been threaded quietly into the domain even had someone wished to. What that does not bound is the other form of the objection — that a domain's *surface* may have been chosen with the known stagings in mind — and no property of a build graph rules that out. Nor would another client on this same domain settle it, which is worth saying because it is the obvious next move and the wrong one: the objection is about who authored *this* domain, so what answers it is the arrangement applied to a domain its author did not design for it. That is a second worked domain, not a seventh client, and this paper does not have one.

A further limit is now measured rather than suspected, and it narrows the result: invariance across *transports* is not distinctive of this arrangement. A ports-and-adapters version of the same game, built and counted (Appendix A, Lab F), also takes a new transport without touching its domain — twice over. So the zeros of Experiment A, taken as counts per staging, do not separate this arrangement from a competent ported one; what separated them in measurement was a new *capability*, a game outliving its process, which cost the ported domain a change and cost this one nothing. The claim to carry forward is about capabilities a record supplies, not about stagings as such. And the distribution stagings —

the two coordinated actors and the three machines — were measured against no baseline at all, since a ported design has no distribution story to port; those zeros stand uncontested rather than confirmed.

That correction has a limit of its own, and it runs the other way. Every other limit in this section narrows what the paper claims; this one narrows the concession just made. The baseline is the author's own. It was written after the fact, by someone who already knew which property it had to exhibit, and written deliberately clean — its eleven rule files differ from the journaled domain's by one line each, the namespace. What it establishes is therefore what ports and adapters *permit*: a rule model with no project references, no packages and no framework named, unmoved by a new transport. It is not evidence that such domains are what the pattern yields in practice, and nothing of that kind is claimed here in either direction. So the comparison does its job — it shows this paper's per-staging counts are not distinctive, and that correction stands — while establishing nothing about the population of ported systems, which would take a study of a kind not attempted here. A reader inclined to read Lab F as showing that the arrangement argued for is already common should note that it shows only that a competent instance of the alternative can be built, by the same author, for the purpose; which is the same standing this paper's own artifact has, and no more.

Two narrower limits belong here as well. The demonstration is one domain on one framework, and it is an existence proof, not a measurement: it shows the separation is buildable and that its mechanical cost is a set of edits not made — a domain diff that stays empty — not that the separation is worth its apparatus at scale, or cheaper in production, which are questions for other work. And the distributed staging of §2 is bounded as that section stated: its peers meet through an out-of-band bootstrap, its coordinator role does not rotate, its transport trusts a certificate on first use, and its replication reaches agreement through a catch-up path rather than an uninterrupted stream. Each bounds the deployment; none touches the domain's invariance, which is the only thing the staging was there to test.

One of those two observations bears on this paper's own claim and so belongs here rather than with them. A fact a domain emits must be derivable within a single actor's state, and that is a property of the substrate constraining a modelling decision *inside* the domain — influence running against the direction this paper measures. It declares no dependency, imports nothing, and its diff stays empty, so §2's measured claim is untouched. What it does mean is that **declaring no dependency on the framework is not the same as being unshaped by it.** The influence is narrow, and it falls on how a domain may be divided rather than on what it says.

One axis is deliberately left aside, and it closes the list of limits. This paper varies where a domain runs and who observes it, and holds the domain invariant across both; it does not treat which actor talks to which — the addressing and topology of a system of many domains — a separate question with its own

answer, not folded into this one.

8.4 Two observations that bound the interpretation

During the experiments two observations emerged repeatedly. Neither is part of the claim advanced here; they are reported because they bound its interpretation and motivate later work.

The first is a hazard, and the evidence for it is the strongest in the paper: **an incomplete record answers plausibly.** Three labs met it independently, by three routes, each taking it for a local incident — a routine correlated to the wrong closing entry with its count still right, a truncated replay returning a board internally consistent and in one case exactly correct, a rehydration silently dropping every zero-argument act (Labs H, I, G). A gap in a record yields not an error but a smaller coherent story. That is unlike *gray failure*, whose defining feature is differential observability, the application's view diverging from the observer's (Huang et al., 2017): here there is none, since every reader reads the same consistent record and no second view exists to disagree. What detects it is therefore not another observer but a ground truth carried forward, which gives a rule cheap enough to state for anyone measuring a journaled system, not this one alone — **a replay measurement must carry the state recorded at play time and assert against it**, or it will pass while being wrong. Two qualifications: it is a rule about measuring rather than running, and the exposure is not peculiar to this arrangement, since any reader reconstructing state from an append-only record has it. It also meets the one row of §9's table of Waldo's four differences marked as not addressed — partial failure of a *peer* is exercised nowhere here, while partial failure of a *record* is what these labs found, and it is quiet in a way a dead peer is not.

The second is a constraint, and it is narrower than the first in a way worth stating before it is read: it is a property of *this substrate's* emitting plane rather than of journaled domains in general, and unlike the first it rests on an argument with checked premises rather than on a measurement — nothing about the loss was measured, because the split actor of that lab exposes no sink, no formatter and no output wiring, so there was no push channel there to lose (Lab G carries both qualifications). On this substrate, then: **a fact a domain emits must be derivable within a single actor's state.** A complete frame was a join over the pile's cells and the falling piece's, computed inside the domain; after the board is cut in two those live in two actors, and a projection on the emitting plane reaches only its own actor's state, so neither role can push a whole frame (Lab G carries the premises and their anchors). Emitting a joint fact therefore bounds the admissible decompositions of a domain — a boundary may be cut wherever §8.2's concurrency test permits, except across a fact the domain must emit whole. What this does *not* touch is §3 or Lab D, whose counts stand as taken, nor observation of a divided domain: a party reading both records and joining them is an ordinary adapter, which is what §3 argues for. That reader was not built and nothing is claimed for it. What closes is the push path.

9. Related Work

The separation of a domain from the machinery that runs it is not a new aspiration; reading it as a *measured invariance* rather than a design prescription is where this paper sits among its neighbours.

9.1 What kind of claim the concessions are against

What follows concedes each of this arrangement's mechanisms to prior work, and every concession is meant literally. But prior art on the parts of a configuration is not prior art on the configuration, and that should be clear before the concessions begin. An architecture in the foundational account is three things and not one — its elements, the *form* that constrains them, and the rationale for that form — where form is the properties of and relationships among the elements (Perry & Wolf, 1992). Same elements under a different form, different architecture. A form is what is claimed here: a domain that declares nothing on either side, whose acts are recorded by a substrate it does not name, staged by parties that name it. Each element of that is old and this section says where each comes from.

The form has four constituents, and because the contribution is the form and not the parts, they are set out together here rather than left for a reader to gather from the places each is mentioned.

	Constituent	Credited to	In the baseline?	What it accounts for
i	a rule model that declares no dependency	information hiding; functional core, imperative shell	**yes** — measured tie	nothing distinctive
ii	entered by calls on its own operations, calling nothing outward	inversion of control	**yes**	nothing distinctive
iii	declares nothing about what becomes of what it emits	implicit invocation	no	the empty publicly callable surface

	Constituent	Credited to	In the baseline?	What it accounts for
iv	its acts are recorded as it performs them	system prevalence; event sourcing	no	history and durability at no cost to the domain

Table 1. The four constituents of the arrangement, what each is owed to, whether the ported baseline of Lab F has it, and which measured difference each accounts for. Every constituent is old; the claim is the set.

Whether the *form* is old was settled by measurement rather than by assertion in a related-work section. The baseline holds the first two and lacks the last two, and each surviving difference traces to one of the two it lacks. That the domain has no publicly callable surface follows from (iii): a port must be visible outside the domain that declares it, and where nothing is declared there is nothing to expose. That history and durability cost the domain nothing follows from (iv) and not from (iii) — a substrate could carry a domain's emissions to a sink without recording any of them, and then a restart would find nothing to replay. The record is what makes the second difference, and it is named as a constituent for that reason.

9.2 Work that isolates a domain

Most of that work can be compared on one property rather than mechanism by mechanism, and comparing it that way is what the construct above buys. **Existing work isolates a domain. None of it asks whether the domain is finished.** Isolation is a relation between a domain and its surroundings, and every line below achieves some version of it; completeness is a property of the domain alone, and no line below measures it.

Prior work	How it isolates a domain	Asks whether the domain is complete
information hiding (Parnas, 1972)	encapsulate the decisions most likely to change	no — a principle to follow, not a property to check
domain-driven design (Evans, 2003)	a domain layer kept clear of infrastructure	no — and its repository is an obligation the domain declares

Prior work	How it isolates a domain	Asks whether the domain is complete
functional core, imperative shell (Bernhardt, 2012a, 2012b; Peyton Jones & Wadler, 1993)	purity: effects confined to a shell	no — its subject is testability, and it is conceded below
model-driven architecture (Kleppe, Warmer, & Bast, 2003; Mellor & Balcer, 2002)	platform independence, by transformation	no — what runs is a derivative, not the model
twelve-factor (Wiggins, 2011; Kratzke & Quint, 2017)	configuration separated from code	no — its subject is the deployment, not the domain
software product lines (Clements & Northrop, 2001; Apel et al., 2013)	one codebase, a variant per configuration	no — it manages intended difference
ports and adapters (Cockburn, 2005; Martin, 2017)	declared ports, adapters outside	no — the ports *are* the incompleteness
location transparency (Hewitt et al., 1973; Agha, 1986; Armstrong, 2003)	an actor reachable without regard to where it runs	no — its subject is the address of a peer
command-query segregation and event sourcing (Meyer, 1988; Young, 2010; Fowler, 2005)	a view not served from the write	no — the mechanism §4 stands on, not a claim about the domain

Table 2. Prior work on isolating a domain, against the question this paper asks of one. Three rows are also concessions of mechanism and are developed below: ports and adapters, because it is the comparison the result must survive; functional core, because it reaches the same zero test doubles; and the two that own this arrangement's input and output edges. Two borrowings of vocabulary are acknowledged rather than compared — *accidental* is Brooks's (1987), used here of the staging while his distinction partitions difficulty rather than identity, so nothing reported here is a silver bullet; and the theatre is Laurel's (1991), whose subject is the audience's experience where this paper takes only the distinction between a play and its staging. Brooks's cut was later applied to software's own materials by Moseley and Marks (2006), separating essential logic from the accidental complexity that state and control flow impose on it — made within a program where this one falls between a domain and its stagings,

and the reason the concession to functional core below is owed at all: keeping an essential part free of accidental machinery is an old goal with a literature.

That tradition also states the isolation goal this paper measures, and it is worth naming the one point where its own answer stops short, since a reader who practises it will feel that point before any other. Domain-driven design asks that the domain layer be kept clear of infrastructure, and for persistence it offers the repository: the interface declared *in the domain layer*, the implementation outside it (Evans, 2003). That is a compromise rather than an oversight, and Lab F shows why it is forced — reconstituting an aggregate requires a way in, so a design of that shape must have the domain offer something. In the vocabulary above, the repository is an obligation the domain leaves open, and a domain-driven design domain is decoupled rather than closed. This one offers no repository interface, no factory for a mapper, and no restore constructor, because reconstitution is the recorded acts being performed again through the domain's own operations: the way in is the way it was always entered.

The scope of that difference should be stated narrowly, because it is easy to inflate into a claim about the whole tradition. It concerns *isolation from infrastructure* and nothing else. Domain-driven design is also a discipline of modelling — a language shared with domain experts, contexts bounded deliberately, a model refined against the business — and a Tetris board demonstrates none of that. Nor does this paper reach the domains that tradition is usually brought in for, which act on the world and then reason about what happened; §8.1 concedes them. What is claimed is one axis: on the isolation goal both share, the arrangement here pays no repository, and the ported baseline built for comparison shows what paying one costs.

9.3 Ports and adapters, and what closure adds to it

Closest, and the comparison the result must survive, is hexagonal architecture — ports and adapters (Cockburn, 2005) — and its restatement as clean architecture, the domain at the centre with every dependency pointing inward toward it (Martin, 2017). The comparison is usually put as a question about staging invariance, and put that way it is settled and uninteresting: a ported version of this same game was built, given the same stagings one at a time, and counted, and two of its transports cost its rule model nothing at all (Appendix A, Lab F). Per staging the two arrangements are indistinguishable, and this paper concedes it without reservation.

The difference that does separate them is not a count of edits, and it is easier shown than argued. Set what each arrangement still requires of someone else side by side, taking every row from Lab F or Lab E:

	What is measured	Ported baseline	Here
I	**outward obligations the domain declares**	**3 driven ports** — board output, piece selection, state	**0**
↳	stand-ins its own tests require	3, without which 20 of 64 tests do not run	0
↳	publicly callable domain operations	at least one per port	0 — one type with no members
↳	a state admissible from outside	`GameState`, arriving through the state port	none; only acts
II	**a reconstitution surface of its own**	**a restore constructor and a pile factory: 56 lines added, 5 removed**	**none** — replay re-enters by the operations the acts were performed through

Table 3. What each arrangement still requires of someone else — **two independent measurements, not five.** The rows marked ↳ are consequences of the fact above them and are shown because each is separately checkable, not because each is separate evidence: a stand-in is a driven port seen from the test side; a port must be visible outside the domain that declares it, so where nothing is declared there is nothing to expose; and the state the baseline will accept arrives through one of those same three ports, the one its own source calls *a third driven port, added by staging 4*. The quantity is unfulfilled obligations, not lines of code.

The conjunction of those two measurements is what this paper calls **closure**, and the word is doing no work the counts do not: *the domain declares no obligation outward, and its reconstitution requires no surface of its own.* A port is an obligation the domain itself declared and left for someone else, so the first fact is what makes the three rows under it follow; and reproducing by the operations the acts were performed through is what makes the second one hold, since there is no event distinct from the act for a separate replay surface to take. A domain with neither obligation is finished. Ports and adapters *decouples* a domain; this arrangement *closes* one.

Two established senses of the word meet here, and both are better dealt with than left for a reader to supply. One is favourable and is claimed deliberately: in

the language of modules and linking a component with no unresolved external reference is *closed*, and the closure of a set of modules is what a linker computes when nothing outward remains to be found. That is very nearly the property measured above, arrived at by asking a different question, and the resemblance is not a coincidence to be excused. The other is unfavourable and is denied. In the open/closed principle, *closed* means closed to modification (Meyer, 1988) — and this domain is not that. It grows: a score and a difficulty level were added to it, additively and without a signature changing (§8.2, Lab I). What is closed here is the set of obligations the domain leaves to others, not the set of changes its author may make to it.

Three things bound that, and the first is the most likely to be misread. Closure is not visible in the domain *considered in isolation*: a hexagon's driving port is the domain's own API, which nobody supplies and nobody doubles, and a domain needing nothing outward declares no driven port either — the `Well` as bare rules over a board is such a domain (§8.1). All five rows appear only once the domain is *deployed*, when its state must outlive a process and arrive on a second machine. In the ported arrangement those became declared needs; here they were never needs, the acts having been recorded as they were performed.

The second bound is on the word. Calling the sum of those counts *closure* is a reading of them, as §5's *identity* is a reading of the structural property, and the paper keeps reading and measurement apart here as it does there. The third is that history and durability are one row of the table and not the claim; purity, conceded below to functional core and imperative shell, buys freedom from effects and closes none of these rows. The obvious objection — that orthodox practice persists an aggregate through an adapter-side mapper without touching the rule model at all — is answered by measurement rather than by argument: the baseline attempted exactly that design and still needed a seam authored inside its own rule model, for a reason its source states and Lab F sets out.

One measured result runs the other way and belongs in this comparison rather than in its appendix. The build graph does not separate the arrangements: a ported rule model is equally free of project references, packages, and framework names. So the testability argument above must be narrowed to what it can bear. What cannot be exercised without a stand-in is the ported *application service* and its three driven ports, without which twenty of its sixty-four tests do not run; the ported *rule model*, taken alone, is as testable and as dependency-free as the journaled domain. The driven and driving sides are not symmetric.

It is fair to press the point: if the substrate provides the mechanism, has the port not simply moved into the framework — universal, unnamed, but a port all the same? Two answers, and the second is the one that can be checked. A term stretched that far stops separating anything: a domain declares neither the garbage collector, nor the scheduler, nor the collections library it computes with, and calling each of them a port empties the word. What *port* earns its keep by naming is an interface a component **declares and depends upon**, because that is the thing with the consequences this paper counts — an implementation

per staging, a stand-in per test. And measured there the difference is not interpretive: the domain of §2 compiles and passes its tests with the framework absent from its build graph, which a domain that declares a driven port cannot do, since a port with neither implementation nor double is not something a test can run against at all.

```
Ports and adapters:
   Adapter --calls--> [driving port = the domain's OWN api] --> Domain
    Domain --declares--> [driven port] <--implements-- Adapter
    (only the driven port is a dependency of the domain, and a domain
     that needs nothing outward declares none)

Here:
    Domain        Staging --binds--> input source / sink / client
   (no declared dependency either way; the state that must outlive the
    process and reach a peer is the substrate's doing, not a port the
    domain asks for)
```

The contract has not been inverted; it has moved off the domain's boundary, and two places are involved rather than one. The *contract* is the substrate's: it defines what an output is, what may command the domain, and how either is carried. The *binding* is the staging's — which sink, which client, which transport — and it happens later than a reader is likely to assume, a destination being attached to a live actor rather than fixed at build or deployment (`Choreography/Theater/PerformanceV2.cs:415`). So the formulation used throughout is that the contract is provided by the substrate and bound in the staging, never that it moved into the staging, which would credit the staging with a mechanism it only wires up. The same shape holds for the transport carrying an actor's messages to another, the domain naming no wire there either; that belongs to the axis this paper sets aside (§8) and to Paper 4, and is mentioned so the output side is not read as a special case. What the move costs is measured rather than reasoned (Appendix A, Lab F): a ported arrangement pays no domain edit per staging either, and its driving side needs no double, a hexagon implementing its driving port rather than depending on it. Where it pays is on the driven side — three ports without stand-ins for which twenty of its sixty-four tests do not run — and in its publicly callable surface, at least one operation per port against none here. That is the observable trace of the move and not the point of it. The point is that the boundary the contract sits on is no longer inside the domain, which is what lets the domain be the common ancestor of §5: a port is a thread from the domain back to the shape of its clients, a fragment of a staging lodged inside it, and while it is there the identity is not clean.

9.4 Who owns the two edges

The geometry itself has a prior claimant, and it is the nearest neighbour this paper has. Implicit invocation — a component announces an event, components

registered elsewhere are invoked, and the announcer names no recipient and declares no interface for one — was formalized as a design space three decades ago (Garlan & Notkin, 1991) and studied as the means by which independently built tools are integrated and independently evolved (Sullivan & Notkin, 1992); its later forms are surveyed as publish/subscribe (Eugster et al., 2003). A domain that emits facts under logical names and says nothing about who receives them is doing that, and the coupling that remains — agreement on the names — is what that literature calls nominal coupling. The geometry of §2 is therefore implicit invocation's geometry, and the honest statement is that this paper did not discover it.

Three things distinguish what is claimed here, and all three are narrower than the geometry. The first is the direction the vocabulary faces. In implicit invocation the event vocabulary is an *integration* vocabulary: announcer and listeners agree on a set of events in order to be integrated, and the names exist because something will listen. The names a domain emits here are its own — the operations it performs and the facts they leave behind — and they would stand unchanged if nothing listened at all; the staging adapts to them, never the reverse. The second is that implicit invocation is offered as a mechanism, with a design space to explore and trade-offs to weigh, whereas the claim here is an invariance measured on a built system: the domain's diff across five stagings and six clients. The mechanism makes that zero possible; it neither asserts nor measures it. The third is the conclusion drawn. That literature concludes that components may be integrated and evolved independently. This paper concludes that the domain stands as the common ancestor of every staging of it — a claim about what a domain is, not about how the parts of a system are connected.

The input edge has a prior claimant of the same kind. That a framework calls the application's code rather than the reverse was named *inversion of control* when frameworks were first described as a form of reuse (Johnson & Foote, 1988), and it is the mechanism by which a domain can be driven without declaring an interface for whatever drives it: the substrate invokes the domain's operations, and the domain calls nothing outward. Section 2's input-side zero rests on that mechanism and the credit belongs there. What the term later came to name in practice — a container that wires declared dependencies together (Fowler, 2004) — is a different thing, and the distinction matters for the reason Paper 8 gave: injection presupposes something to inject, and where a domain declares no port there is nothing to wire. The mechanism is inversion of control; the measurement, and the identity it reveals, are what is added here.

One practitioner pattern has arrived at that same geometry from a different motive, and it deserves an explicit concession. Functional core, imperative shell places a dependency-free core inside a shell that owns all input and output, and its best-known consequence is precisely the count reported above: the core can be tested with no test doubles, because there is nothing there to double (Bernhardt, 2012a, 2012b). That zero is therefore not this paper's discovery, and

it should not be presented as one. The name belongs to practitioner literature, but the discipline beneath it does not: confining effects to a boundary so that the remainder can be reasoned about and exercised without them is the monadic treatment of input and output, argued in the refereed record three decades ago (Peyton Jones & Wadler, 1993). What differs is the claim the zero supports. The pattern buys its core's independence with *purity* — the core is functions without effects and the shell holds the mutation — and it is a discipline about the internal structure of a single program, whose shell is one imperative wrapper. The domain here is not pure: it mutates. What is recorded of that mutation is recorded by the substrate, and the domain never learns a journal exists — which is the measured claim of §2 and not a courtesy of this comparison. So purity is neither available as the mechanism nor needed as one: the independence is bought not by declining to mutate but by keeping the account of the mutation someone else's work. And its shell is not one wrapper but many stagings, one of them spread across three machines. Same zero, different result: the pattern explains why a *test* needs no double; this explains why a *staging* needs no domain edit, and what that says about the domain's identity.

9.5 The local/remote objection

Which raises the strongest objection available to this paper, and it is an old one. Waldo, Wyant, Wollrath, and Kendall (1994) argued that the distinction between local and remote cannot be papered over: latency, memory access, concurrency, and partial failure are irreducible, and systems that hide them fail precisely where reliability is demanded. Experiment A — a domain moved from one process to three machines with no edit — has the shape of the thing they warned against. The objection is accepted here rather than rebutted. Nothing in this paper claims those differences vanish, or that distribution is free. Nor does it help to gesture at the incidents a distributed staging happens to produce — an out-of-band rendezvous, a connect-readiness race, a certificate trusted on first use — as though each answered one of Waldo's four. It is more useful to set the four out and say, for each, where it falls here and whether this paper does anything about it:

Waldo's difference	Where it falls here	Status
Latency	the transport, and the replication that crosses it	present; nothing about it was measured, no timing was taken
Memory access	nothing is shared by reference at all: each node keeps its own copy of the record and reconstructs its own state from it	answered by the arrangement rather than borne by it

Waldo's difference	Where it falls here	Status
Concurrency	the connect-readiness race of Lab B; one writer per record	present, and the race is reported
Partial failure	not exercised — no peer was killed, no partition induced (Lab B)	**not addressed**

Table 4. Waldo et al.'s four differences against what this paper does with each. Two incidents a reader might expect to find here belong to neither column: an out-of-band rendezvous is a bootstrap question and trust-on-first-use a security one, and neither is among the four.

Read that way the honest statement is narrower and more interesting than the one it replaces. *Memory access* is the case where it hurts most, and it is not evaded: there are three copies of the board's state in Lab B and no shared reference anywhere, which is not a workaround for the absence of shared memory but an arrangement that never assumed it. *Partial failure* is where this paper is weakest, and Lab B says so already. What holds is that the differences which do appear are borne by the staging, and that none of them reached the `Well` — a claim narrower than location transparency's, and deliberately so: the local/remote distinction is real and irreducible, and it is not a fact about the domain.

9.6 How the claim is checked

The method by which the claim is checked has a literature of its own. Software reflexion models compare a high-level model of a system against the dependency graph extracted from its source, reporting where the two converge, diverge, and fall silent (Murphy, Notkin, & Sullivan, 1995, 2001). The conformance-checking work that followed made such comparisons routine: languages in which permitted and forbidden dependencies are declared and then checked against the code (Terra & Valente, 2009), comparative accounts of the available static techniques (Passos et al., 2010), and tools — ArchUnit, NDepend — in which such rules are asserted as ordinary unit tests. What motivates that whole literature is architectural erosion: the drift of a built system away from the architecture intended for it (Perry & Wolf, 1992). Section 2's reading of the build graph is a measurement of the same kind, and the resemblance is what lets this paper say *checkable* rather than *arguable* — with erosion as the instructive contrast, since it is what a dependency direction does when nobody is enforcing it, and the empty diff is what the same direction looks like when it is a property of the arrangement rather than a rule someone must police. The difference lies in what is checked. Conformance checking takes an intended architecture as its input and counts departures from it; there, the direction of dependence is a rule

imposed and then policed. Here the direction is not a rule but the substance of the claim, and the quantity measured is not a violation count but an invariance under deliberate variation: change the staging, and see whether the domain moves. The first is an audit; the second is an experiment.

9.7 What §4 owes the data literature

The nearest neighbour of §4 is a data-management literature, and it must be credited carefully, because two distinct papers bear on it. *Immutability changes everything* argues for append-only computing: observed facts recorded and kept, results derived from them on demand, on the model of an accountant who corrects by adding an entry rather than by erasing one (Helland, 2015); Kleppmann and Kreps (2015) turn that into a way of composing systems around a log. The earlier paper is the one that reaches §4's temporal claim. *Data on the outside versus data on the inside* holds that data outside a service is beyond the scope of that service's transactions and temporally disconnected from it (Helland, 2005) — which is to say that what an external consumer holds is always the past. Section 4 therefore does not discover that; it arrives at it twenty years later, in the vocabulary of roles rather than of services, and the honest statement is that the data literature drew the conclusion first.

What survives the concession is narrower, and worth stating exactly. Helland's argument concerns where data sits relative to a transaction boundary and what may be assumed of it there. Section 4's concerns *offices*: that watching and acting are distinct roles; that the present is available only in the acting one; and that an audience is therefore constituted by the record rather than merely served from it. Its operative consequence is a design corollary the data framing has no occasion to state — a command's response cannot serve a client that issued no command, so a system whose only view path is that response has none for an observer — which follows not from a property of data but from whose the response is. The temporality is Helland's, and the structure beneath it Lamport's; the attribution of roles, and the corollary it yields, are what this section adds. It does not claim more than that, and §4 says why: returning state in a command's response is a recognized session guarantee (Terry et al., 1994), not an error, and the present is observable by an audience that polls for it.

9.8 The series, and what remains unclaimed

Two of this series' own papers hold pieces of the present result. Paper 7 read the server as an *accidental category* — ephemeral under this substrate, discharged at bootstrap and absent from the running topology after — and offered that as a lens rather than a result. The consequence drawn here is that where a domain runs is then not constitutive of it, which is one axis of the staging this paper varies. Paper 8 located, from the side of output, the boundary between what a domain holds and what an actor projects for a client, which is the adapter boundary of §3 seen from within a single output. This paper generalizes both:

it varies the whole staging — where a domain runs and who observes it — and names what stays fixed across all of it as the domain's identity.

What remains unclaimed, after all of these, is the intersection — which is to say the form, in the sense this section opened with. Late binding of a destination is old; platform independence has a name and a methodology; location transparency has a production tradition; a dependency-free core has a pattern; being driven without declaring an interface for the driver has a name three decades old; and checking a dependency graph against an intended model has a literature. Each of those is conceded above, individually and without reservation. What none of them states is that the contract may leave the domain on *both* sides at once, so that a domain declares nothing about what drives it or about what it emits — and that the property this leaves behind, the domain standing as the common ancestor of every staging of it, is measurable after the fact on a system already built.

A reader is entitled to ask why a list of conceded parts should leave anything behind, and the answer is the one the section opened with rather than a rhetorical one. The claim is about a form, and a form is tested by building the alternative. The alternative was built, from parts every one of which is credited above, and it produced neither of the two differences that survive. Nothing here rests on the parts being new. What it rests on is that assembling them the ordinary way does not arrive here — a claim about the arrangement, and one this paper publishes a falsification test for: build a ported design that obtains durability and a second machine without touching its rule model.

One such design is available and needs naming, since it is the first a reader who has built one will reach for, and it is nearer than the mapper of Lab F. An event-sourced hexagon declares an event-store port and reconstitutes by replay rather than by mapping, so it needs neither a restore constructor nor a factory for a pile, and the 56 lines this paper counts would not be spent. What it does need is a *reproduction surface* distinct from its command surface: an orthodox event-sourced aggregate exposes something of the shape of `Apply(event)` and `LoadFromHistory(events)`, so the rule model still grows a seam, only a different one. The residual claim stated exactly is therefore not that a ported design must pay a mapper's seam, but this: reproducing by the domain's *own* operations requires no reconstitution surface at all, because the substrate re-invokes the very verbs the acts were performed through, and there is no event distinct from the act for an `Apply` to take. That is *predicted* rather than measured — no event-sourced baseline was built, and building one is the lab that would close the test this paper publishes.

10. Conclusion

This paper began where practice is settled. A program's domain and its deployment are written together, so changing where a system runs, or which client it serves, means changing the system. Set beside theatre, where one script is

staged in many venues without being rewritten, that entanglement stops looking like a property of software and starts looking like a confusion of a play with its staging.

The confusion has a measurable opposite. One domain — a Tetris board — was run on a console, in a browser over a WebSocket, in a browser over server-sent events, across two co-hosted actors over real TLS, and across three separate machines in containers; and it was read by a person at a keyboard, an automated player over a pipe, two passive observers, and a browser drawing it in JavaScript. The number of changes that variation forced on the domain is zero, and the number of test doubles the domain's own tests require — for what drives it or for what it emits — is also zero.

Behind those counts is a property of the domain rather than a discipline, and the counts are not where it lies: a ported rule model is equally free of references and packages, equally testable without stand-ins, and equally unmoved by a new transport (§9), so those counts distinguish a clean domain from an entangled one and not this arrangement from that one. **Ports and adapters decouples a domain; this arrangement closes one.** Section 9 puts that on two measurements — whether the domain declares any obligation outward, and whether reconstituting it needs a surface of its own — and the 61 changed lines a ported rule model spent to survive a restart are one row of the second, not a separate result. The domain here declares no interface for what drives it or for what it emits, so the contract fixing what an output means, where it goes, and what may command it does not sit on its boundary at all: the substrate provides it and the staging binds it. That is not dependency inversion carried a step further; it is the contract in a different place, and it is what leaves every staging's references leading to the domain and none leading out — a sink in the dependency graph, the root of its reverse, which is all the paper means in calling the domain their common ancestor. What is checkable against a built artifact is that structural property together with the empty diffs; reading it as an *identity* is what §5 does, on the terms §5 sets. Read so, *precedes* names the direction of the dependence. A staging is built against a domain that does not know of it; the domain is not the thing that survives its stagings but the thing that makes them possible.

Two consequences followed with no new machinery. Watching and acting are distinct roles, and what each may have differs: a live read taken *in order to act* is part of the act and belongs to the party acting, while a view served from a command's response can serve no client that issued none. So an audience is constituted by the record rather than served a state read off the moment — not because the present is closed to it, since one of the six clients polls the board and is handed it, but because what an audience is told is an account and not a configuration. And because the domain keeps its identity, the account of what it did keeps its identity too, so the routine those acts compose is recognizable from any stage, through any adapter. Neither is a second claim; both are what the first one entails.

What can be carried away is an instrument rather than a prescription, and the difference is worth being exact about, since this paper declares itself the analytic kind of contribution. Nothing here establishes that an architecture is better for holding the property, which would be a judgment about what makes an architecture good. What the paper supplies is a question that can be put to a system already built and answered by measurement rather than opinion: does the domain inside it hold an identity independent of where it is staged and to whom? Vary the staging, and count what the domain had to change. The labs of Appendix A are that question asked once, of one domain, with the answer zero.

Two things came out of those labs that nobody set out to find. Neither is the result of this paper, and both are reported because they bound how its result may be read — the first by bounding what a reader of a record can be told, the second by bounding how a domain that keeps one may be cut. The first is a hazard, and the evidence for it is the strongest in the paper: **an incomplete record answers plausibly.** Three labs met it independently, by three routes, none investigating it — a routine correlated to the wrong closing entry with the count still right, a truncated replay returning a board that was internally consistent and in one case exactly correct, a rehydration silently dropping every zero-argument act. A gap in a record yields not an error but a smaller coherent story, and because every reader reads the same record and it is consistent, no second view exists to disagree. The rule that follows is cheap and is offered to anyone measuring a journaled system, not just this one: carry the state recorded at play time and assert a replay against it. A replay checked only for internal consistency will pass while being wrong. The second is a constraint, and it is stated in §8: a fact a domain emits must be derivable within a single actor's state, so emitting a joint fact bounds how that domain may be divided — the one respect in which the substrate shapes the domain rather than the other way round, and the more interesting for being the exception. Whether an architecture *ought* to have the property — whether the arrangement pays for itself, and against what — is exactly what §8 and Appendix A decline to claim, and a reader who wants that will have to measure it.

Four questions the argument opened are left standing, each larger than this paper. If an entitlement must be attributed to something that holds still, then identity is prior to authority, and the series' earlier questions about who may speak rest on this one. Who is *entitled* to read a routine, §6 reaches without settling — though whether a reading of one can be wrong it does settle, and the answer is that it can, and silently, when a moment that mattered was never an act. Which is the third question, and the one this paper is most exposed on: what must be an act at all. A criterion is offered here for a record's vocabulary and none for its granularity, and it is granularity that an account's fidelity rests on (§6). And because a domain that declares no dependency cannot reach for another, whatever composes two domains is neither of them — the question that anything treating a set of domains as a repertoire would have to answer first. Closure is what this paper hands to that question. A domain that still declares

obligations is a *part* of a system, completed by whatever supplies them; one that declares none is a whole, and a whole is the only kind of thing a repertoire can hold. Whether closure is sufficient for that, or merely necessary, is not settled here.

The audience changes. The stage changes. The play asks nothing of either.

Appendix A. Labs

The claim of §2 is a count, and this appendix reports the counts and says where each can be checked. These labs are illustration rather than evaluation: each shows that a separation the paper argues for is *built*, and what it costs the domain, which is nothing. In the manner of the earlier systems papers, every result here is a count the separation drives to zero, and the zeros are the claim. Two things are not measured and are marked as such rather than mixed in: whether the arrangement is cheaper or faster at scale, or better in production, which §8 leaves open; and what the distribution zeros are zeros *against*, since the one alternative arrangement that was built — the ported baseline of Lab F — has no distribution counterpart to set beside them.

All of them run against one domain — the `Well` and its operations (`domain/`) — reached by every host through a single actor (`actor/TetrisActor.csproj:11`), which the example's own comment calls an accidental shell (`actor/IGameHost.cs:14`). The domain project declares no reference of its own (Lab E).

Lab A — the stage. The same `Well` runs on a console; in a browser with input and output carried over a WebSocket; in a browser over HTTP with server-sent events; and across two StageManager actors, joined once in memory and once over a real Kestrel TLS channel. Five stagings, differing in process, transport, and wire format. A diff of the domain directory from the first staging to the last is empty, and the example builds with no errors. The domain's own test suite also passes unchanged throughout, which is worth stating for what it is and no more: a regression check showing the stagings did not break the domain. Its coverage was not measured, so neither the suite's size nor its passing is offered here as evidence that the domain is adequately exercised — a test count is not a coverage measurement, and nothing in this appendix should be read as one.

Lab B — three machines. Three Docker containers run the same `Well` as three StageManager peers on a private bridge network: one Director, two casts, joined over Kestrel TLS on a port never exposed to the host, with coordination and replication crossing genuine container-to-container TLS. A scripted driver on the Director plays the game to a non-trivial board; the three nodes converge on the same journal entry and the frame each pushes is byte-identical. Adding this staging left both the domain and the actor untouched — the diff of each across the whole increment is empty, and the files added are a new host, its Docker definitions, and notes.

Two details of that run belong here rather than in the body. The game was played by a script rather than by a person, but it was played: the `Well` genuinely mutated on one machine and the same state reached the others. And what each node is handed is the domain's *assembly*, named through the anchor type — the `Well` itself is passed nowhere and constructed by no host, but built inside each node by a recorded act, the seeding command the actor issues on startup, so that a board exists on three machines without any of them ever having held one.

The weight this lab can carry is small, and saying so is more useful than dressing it up. It is a *single* run, reproducible from one script but not repeated, with no fault injection: no peer was killed, no partition induced, no message dropped. "The three converge" is therefore an existence result and not a claim about convergence under adversity, which repetitions and induced failures would be needed to support. Its other boundaries are the ones §2 states — an out-of-band rendezvous, a fixed Director, trust-on-first-use certificates, convergence completed by the framework's catch-up path. And by this paper's own §8 the lab is measured against no baseline at all, since a ported design has no distribution story to port. What it establishes is that the domain's diff stays empty when the staging is genuinely cross-machine, which is one more staging on the same axis as Lab A rather than a second kind of evidence.

Lab C — the client. With the stage held fixed, the client varies: a person at a keyboard reading an on-screen grid; an automated player sending moves over a pipe and reading a view it computes for itself; two passive observers, one pulling the board and one receiving it pushed; and a browser in which a player and a spectator are both written in JavaScript and reach the game over the network. Six clients over one `Well` — some in C#, one a PowerShell script, two in a browser — and the domain directory does not change for any of them. Each adds an input adapter, an output adapter, or both. One of those adapters was written by the client that reads through it rather than by the author of the domain: the automated player is an instance of a large language model and authored its own column-height view during the lab (§3; disclosed in the acknowledgments).

Two counts belong here rather than one, because they answer different questions and the larger flatters the smaller. Six is the number of clients added, and it is the number that bears on the domain's diff: each was a separate addition, and none of the six cost the domain an edit. But six overstates how *differently* the board was read. Three of them — the person at the keyboard, the observer that pulls, the observer that receives a push — read the same grid over three different transports, and the browser's grid is that grid once more, computed independently in JavaScript. Counted by representation instead of by client, these labs hold **two**: a grid of cells, and a vector of column heights. Counted by implementation, three: a C# renderer, a JavaScript one that mirrors it, and a PowerShell height profile. The six is what supports the invariance claim; the two is the honest measure of representational distance, and §3's argument that

no view is privileged rests on the two and not on the six. Both are readable off one table, so a reader need not take either count on trust:

Client	Reaches the domain by	Representation it reads
console, driven by a person	keyboard, in process	grid of cells (C# renderer)
automated player	a pipe to a warm server	vector of column heights (PowerShell)
observer	a query it issues, pulling	grid of cells (C# renderer)
watch	a pushed frame	grid of cells (C# renderer)
browser player	WebSocket	grid of cells (JavaScript)
browser spectator	server-sent events	grid of cells (JavaScript)

Table 5. The six clients of Lab C. Reading down the last column gives the count that matters: two representations, one of them implemented twice in different languages. Reading down the middle column gives six distinct ways in, which is what the domain's empty diff was measured against.

Lab D — the adapter. One emitted frame reaches three different projections, none of them the domain's. The board emits its occupied cells — a set it computes, in its own vocabulary (`domain/Well.cs:322`, and §3 on why that is a fact it holds rather than a view); a renderer lays that out as a terminal grid (`actor/BoardRenderer.cs:19-48`); the browser produces the same grid independently in JavaScript, its code commented as mirroring that renderer (`web/Program.cs:169-178`); and the automated player lifts the same frame into a column-height profile with its gaps, wells, and aggregate figures (`tools/pile-scan.ps1:54-83`). Three implementations over one fact, and no domain method for any of them — but two representations, not three, since the browser's grid mirrors the terminal's; the height profile is the one that differs in kind.

Lab E — both sides testable, with the framework absent. The domain's own tests run with no staging bound at all: no host, no transport, no sink, and no double — not for what drives the domain and not for what it emits. Stronger, and this is the form in which the claim of §2 is checkable rather than interpretive: the framework is absent from the domain's build graph altogether. The domain project is four properties and no references —

The fence at the level of members. Every verb-bearing type in the domain is `internal`: the board, the pile, the shapes, the piece types, the seven pieces, the rule exception (`domain/Well.cs:34`, `domain/Pile.cs:21`, `domain/Frame.cs:29`, `domain/PieceType.cs:9`, `domain/Pieces.cs`, `domain/TetrisRuleException.cs:17`). The library's whole public surface is one empty type, so not even the actor's own project can invoke a domain

operation in the host language — it obtains the assembly and passes it on. The compiler refuses where reflection admits.

Where in-language access was genuinely wanted it had to be granted by name, from inside the domain's own assembly, and it was: to the domain's test project and to one console host (`domain/AssemblyInfo.cs:8-9`). That is an authored exception, enumerable, and extended to no other staging — the four remaining stagings of Experiment A cannot name a domain type at all. Two qualifications belong with it. One grant is to a *staging*, so the claim is not that no staging may call the domain but that calling it takes an act of authorship inside the domain, by name. And that grant is no longer exercised — the console drives the actor and names no domain type, as its own project comment says — so the fence is in practice tighter than its source admits and the grant could be withdrawn without breaking a build. It is reported rather than tidied away to purchase a cleaner claim, and it is left standing: withdrawing it would tidy the fence at the price of changing what this lab measured, which is a poor trade for a count the paper leans on.

It would be wrong to say the domain references *nothing*: it targets a language runtime and uses that runtime's collections, as every C# project does. The precise claim, and the only one this paper uses, is narrower — **the domain declares no dependency on the framework that runs it, or on any staging of it.**

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net9.0</TargetFramework>
    <Nullable>enable</Nullable>
    <RootNamespace>Tetris</RootNamespace>
    <AssemblyName>TetrisDomain</AssemblyName>
  </PropertyGroup>
</Project>
```

— with `dotnet list package` reporting no packages for it, no framework namespace imported by any of its sources, no framework attribute, and no framework interface implemented; the test project references a test runner and the domain and nothing further. The suite passes on that graph — a regression check, as Lab A says, not a coverage claim.

What this count does and does not separate is settled by the ported baseline of Lab F, which has the same clean graph: zero project references, zero packages, no framework named. So *this* count ties, and "zero references, zero packages" separates a clean domain from an entangled one rather than one arrangement from the other. What does not tie is on the driven side only: the ported application service cannot be constructed without stand-ins for its three driven ports, and twenty of its sixty-four tests cannot run without them, while the journaled domain's tests need none.

Lab	What varies	Measured
A	5 stagings (console, WebSocket, server-sent events, StageManager in memory, StageManager over TLS)	0 domain edits; the domain suite passes unchanged (a regression check, not coverage)
B	3 machines (containers, container-to-container TLS)	0 domain edits **and** 0 actor edits; three nodes converge, byte-identical frame
C	6 clients — some in C#, one a PowerShell script, two in a browser; 2 distinct representations among them	0 domain edits
D	3 projections of one emitted frame	0 domain methods for any view
E	domain built and tested with the framework absent from its build graph	0 references, 0 packages, 0 doubles; the suite passes
G	the domain's own internal boundary — one role cut into two, stagings held still	11 of 12 **hosts** unchanged (the 12th, one line); 0 divergences over 47,783 steps; a whole game replayed into both roles, game over included; record 2.32× (316 against 136), 6 extra entries per landing
H	one reaction over 3 stagings' journals (single process, warm server over a pipe, 3 containers read in place)	same 2 placements in the same order in all three; the 3 containers' journals byte-identical, and the acts matching verb for verb across stagings with entry ids offset by a constant; 0 domain edits — with no correlation handle, no landing act, and entry ids not staging-invariant

Lab	What varies	Measured
I	the domain itself grows — a score and a difficulty level added	+30 lines of code (98 added lines in all, 62 doc comments and 6 blank), 0 code lines deleted (3 doc-comment lines removed), no signature or verb changed; build graph byte-identical; 12/12 **hosts** keep running at 0 edits; adoption costs 0 for four, 1 for five, 4 for each browser host, 32 for the input host — only where wanted

Table 6. What each lab measured. Every entry is a count taken from the example's own history or build, and none of these rows is a comparison — the one comparison, against a ported arrangement built for the purpose, is Lab F and is reported separately below.

Lab F — the ported baseline, built and counted. The comparison §9 rests on is a measurement rather than an estimate, and this is where it was taken. An orthodox ports-and-adapters arrangement of the same game was written — its eleven rule files differ from the journaled domain's by one line each, the namespace, so the comparison is not rigged by writing a worse domain — and four stagings were added to it one commit at a time so that each could be counted: a console, a WebSocket host, a REST-and-server-sent-events host, and an automated player. All four run. Adding the baseline changed nothing on the journaled side: the diff of the domain, the actor, the solution file, and all fifteen existing hosts is empty, and the journaled side's tests still pass.

A ported arrangement invites four estimates about what it costs a domain, and measurement contradicts two of them, ties a third, and leaves a difference that is not the one the estimates predict. Table 7 reports each against what was counted.

Estimate	Measured against the built baseline
an implementation per staging	**confirmed** — 2, 1, 1, 2 across the four stagings

Estimate	Measured against the built baseline
a stand-in per driven port	**confirmed** — three; the application service cannot be constructed without them, and 20 of 64 tests cannot run at all
a domain edit per staging	**refuted** — the WebSocket and the server-sent-events stagings each cost the ported rule model nothing, identical to the journaled side, twice over
a stand-in "per side"	**refuted** — the driving side is zero, because a hexagon *implements* its driving port rather than depending on it (§9)
a distinguishing build graph	**tie** — both rule models: zero project references, zero packages, no framework named
cost of a new *capability*	cross-process persistence moved the ported arrangement by 204 lines added and 9 removed, of which 61 changed lines — 56 added, 5 removed — fall inside the rule model itself; the journaled side gained the same capability, for the same client, at nothing

Table 7. The ported estimates, measured. Two are refuted, one ties, and the difference that survives is not the one the estimates predict.

What the baseline actually shows: the unit of account was wrong. A port costs a domain nothing per *staging* and everything per *capability its ports do not already carry*. Two new transports moved the hexagon not at all — the same zero this paper reports, reached by a different arrangement — so invariance across transports is not distinctive, and §2's counts should not be read as if it were. What moved the hexagon was persistence: to let a game outlive its process, the ported rule model had to grow a restore constructor and a way to rebuild a pile from cells — 61 changed lines, added and removed, not added alone. The journaled arrangement acquired the same capability, for the same client, at no cost to the domain, because a record of the acts already existed. The measured difference is therefore not *per staging* but *per capability of the kind a record supplies for free*.

Read as a test of an arrangement rather than as a scoreboard, that is the more useful thing the baseline does, and §9 leans on it there. This lab holds two of

the three constituents of the arrangement the paper argues for — a rule model free of dependencies, and a domain entered by calls on its own operations rather than by calling outward — and lacks the third, declaring nothing about what becomes of what it emits. Holding two of three, it produces neither surviving difference. Which is what makes the paper's claim a claim about the assembly and not about any part of it, since every part is credited to prior work.

One number in that table needs a caveat stated plainly, since it is the most attackable figure in the baseline. Where a ported arrangement's *domain* ends is a matter of layout, and this baseline places its rule model, its ports, and its application service in a single project — the ordinary arrangement at this size, and the one comparable to the single project the journaled domain occupies. Read that way, persistence cost 204 lines added and 9 removed. A reader who counts only the pure rule model should read it as 56 added and 5 removed, and that is the figure worth arguing with, being the part no choice of layout can move outside the domain. Both are reported, either reading is available without recomputing anything, and the choice changes no other measurement in the table — the stand-ins are required by whichever project owns the port. On both readings the comparison is against nothing.

The rival design that would dissolve that number, and what it costs instead. There is a standard move in orthodox practice that would make the 56 lines an artifact of one implementation choice rather than a property of the arrangement: persist through a separate persistence model and a mapper on the adapter side, so the rule model never learns that anything is stored. If that were available here, the paper's residual claim would fall with it. The reason it is not available lies in the model rather than in the choice, and it can be read off the code. `Pile` is `internal sealed` with a `private` constructor (`domain/Pile.cs:21`, `:28`); the only construction paths it offers are an empty pile and the landing transition (`:35`, `:56`); and it enforces one invariant by construction, that a pile never retains a complete row (`:16`).

A mapper in another assembly therefore has three ways in, and each lands somewhere. It can be handed a factory the domain authors, which is a change to the rule model. It can be granted `InternalsVisibleTo` — which reaches `internal` members but not `private` ones, so it still needs a domain-authored factory, and the grant is itself a line inside the domain naming a persistence assembly. Or it can use reflection, which reaches a private constructor and is what mainstream object-relational mappers in fact do; that route asks nothing of the domain, and it bypasses every check the constructors perform.

The baseline took the first road, and its own source records why. Its rule model grew a factory, `internal static Pile Of(int width, ImmutableHashSet<Position> cells)`, whose comment calls it the seam a state port needs to put a saved pile back, added for the fourth staging because "the only way in was through the model" (`baseline-hex/domain/model/Pile.cs:51`). It grew a restore constructor whose comment names the cost outright — the point at which persistence reached the model (`baseline-hex/domain/model/Well.cs:75`).

That constructor re-asserts the invariants on the way in, the complete-row check among them (`baseline-hex/domain/model/Well.cs:305`, `:314`). Its assembly grants `InternalsVisibleTo` to its test suite and nothing else (`baseline-hex/domain/AssemblyInfo.cs`), and both new members are called from inside the domain project (`baseline-hex/domain/model/Well.cs:86`, `baseline-hex/domain/application/GameService.cs:197`), so the mapper never touches the model. The separate-persistence-model design is what this baseline does; it still required the seam.

Stated no more strongly than it should be: the claim is not that 56 lines is the necessary price. A model with public constructors and no invariants would pay less, and would have less to protect. The claim is that where a rule-bearing type is closed and enforces something, putting a saved instance back is a domain-side act — pay for it in the rule model, as the baseline did, or reach past the constructors and forgo the check.

The journaled side does neither, and the reason is qualitative rather than a count. Reconstitution there is re-performing the recorded acts through the domain's own operations, so `Integrate` runs again on the way back in and the invariant holds on the reconstituted pile exactly as it held on the original — no seam, and nothing bypassed. A mapper reconstitutes *around* the invariant or is let in through a door the domain opens; a replay reconstitutes *through* it. What is not measured is the third road: no reflection-based mapper variant was built, so that comparison is an argument with checked premises rather than a count, and building it is the next measurement this baseline wants.

One concession runs the other way, and belongs here rather than in a footnote. The build graph does not distinguish the arrangements: a ported rule model is also free of project references, packages, and framework names, so "zero references, zero packages" should stop carrying comparative weight — it distinguishes a clean domain from a dirty one, not this arrangement from that one.

A second concession was drafted beside it and did not survive checking. It read that the ported arrangement is cleaner on one point, needing no counterpart to §2's empty public anchor type. But the framework's seam takes an *assembly*, not a type (`Choreography/Theater/PerformanceV2.cs:60`), and reads one with `GetTypes()` under a filter that admits internal types (`Puppeteer/EventSourcing/DomainLibraries.cs:136,151`); the framework's own CLI loads domain libraries by full path (`PuppeteerCli/AttachCommand.cs:309`). Rebuilt with the anchor deleted, the domain exposes zero public types, and a host holding no reference to it, loading it by path, seeds a `Well` and spawns a piece to the same reported state. The anchor is this example's ergonomics and not the arrangement's cost, so the asymmetry claimed for it is withdrawn.

What remains unmeasured is distribution. The two StageManager stagings of Lab A and the three machines of Lab B have no ported counterpart here, deliberately: a hexagon has no distribution story to port, so building one

would mean inventing an architecture rather than measuring one. Lab B's zero therefore stands uncontested by measurement rather than confirmed against a rival.

Lab G — re-decomposition. Every other lab holds the domain fixed and varies what surrounds it. This one does the opposite: it cuts the domain's own internal boundary while the stagings stay where they are. The `Well` was split into two roles, one owning the settled pile and one owning the falling piece; the original's record was then read — by the same introspection read that rehydration performs, of each action and its parameters — and the new roles were driven to perform their own. No journal was cut, transformed, or rewritten: the original's acts are identical before and after, with nothing added or removed, and both new records are append-only with contiguous entry identifiers, each holding only its own role's verbs.

The exercise runs over a whole game rather than a prefix of one: 129 acts to game over, 29 landings, 136 entries, every emitted act present in the record. Both roles reach the `Well`'s exact final state, game over included — which is the interesting part, because the re-cut moved the authority for *game over* to the pile role, and replaying the record into the two reproduces it with both agreeing. Two things make that a fair test rather than a private one. The framework's own rehydration of the same journal reaches the same state, so the record is a record in the ordinary sense and the lab's reading took nothing special out of it. And the full length is available at the substrate commit this paper cites, an earlier one having stopped the readable record short of the game's end.

What was measured. Eleven of the twelve hosts were untouched while the domain divided beneath them; the twelfth needed one line. The invariant held over 60 games and 47,783 steps compared against the single-`Well` version — 9,265 landings, 3,336 rows cleared, 39 games ended, no divergence — and 47,783 overlap checks that crossed the new boundary, asking each role separately, found no overlap; the roles never disagreed about a game being over. Those figures were re-run on the corrected example at the substrate commit this paper cites, and reproduced to the digit, so the equivalence result was never sensitive to which state the example sat in; only the entry counts were. A random player cleared no rows at all in twenty games, so two further playing policies had to be written before the collapse was exercised. Nothing became speculative, and two checks came out stronger than they had been.

What it cost. This is the one lab in which the domain does change, by design: `Well.cs` was untouched, `Pile.cs` took two lines, and 292 lines of new domain were written. Movement costs no cross-role message; a landing costs two tells and two acknowledgements. The record costs 2.32 times as much — 316 entries across the two roles against 136 for the same game — and almost all of the difference sits exactly where the two roles have to speak. Of the 180 extra entries, 174 are the 29 landings at six entries each: a landing writes the piece's handover, the pile's absorption, and a message and acknowledgement each way, against the one act the undivided board wrote. One more is the second seed, since

two roles open two aggregates. The remaining five are one-time declarations — eleven templates across the two roles against six for the one, each written once however many times its act recurs. Entry counts depend on how verbs are recorded, and these were re-taken after every command in the example became a parametrized Action: a template plus a compact argument per act, where a literal script wrote one full sentence per call. That encoding is why the original comes to 136 rather than the 130 an earlier run reported, and the ratio has moved with it, from 2.38× through 2.29× to 2.32×. What the encoding does not touch is the shape of the finding: 129 acts, 29 landings, six entries per landing, and no divergence. A fourth state appeared, because landing is now two acts by two parties rather than one; a guard had to be restated; and a cell-set serialization moved inside the domain, a tell carrying ordered scalars. One further consequence is larger than a cost, and §8.4 states what it amounts to; the premises are here, each of them checked. The undivided board built a complete frame by unioning the pile's cells with the falling piece's and clipping to the interior, a derivation performed inside the domain (`domain/Well.cs:322`). A projection on the emitting plane is produced by a reaction belonging to one actor, running on that reaction's thread once that actor's read lock is released (`Puppeteer/IOutputSink.cs:106`), so it reaches that actor's state and no other's. After the cut those two halves sit in two actors, and neither role can push a whole frame. A third premise is a fact about this lab's own artifact rather than about the substrate, and is why nothing about the loss was measured: the split actor exposes no sink, no formatter and no output wiring, so there was no push channel here to lose in the first place. The reader that would join the two records was not built and is not claimed.

The finding nobody anticipated. The record is not neutral between the two new roles. Every act in the `Well`'s journal turned out to belong to the piece role, and the pile role's own act — absorbing landed cells — appears nowhere in the original, because under the `Well` absorbing was never an act at all but a private consequence of a tick. The pile role's record therefore does not translate; it is generated, by the piece role performing and telling. So a re-decomposition does not divide an existing record between two roles: one inherits the original's voice, and the other's record is born as a consequence of it. Whether that holds in general, a single case cannot say.

Lab H — the same routine, recognized on three stagings. One reaction is defined outside the domain: it seeks a spawn, then the drop that ends the piece's descent, and what it matches between them is a placement. It is then run against three stagings — the journal of a single process; the journal a warm server keeps while a client drives it over a named pipe; and the journals of the three containers, read inside each node against its own store. It recognizes the same two placements, in the same order, in all three. The three containers' journals are byte-identical, and the domain's diff for the exercise is empty: a reaction is defined outside the domain and the domain learns nothing of it.

Three measured qualifications matter more than the confirmation. There is

no correlation handle in this record: a spawn names a piece type and a drop names nothing, so the trajectory is tied by order alone. There is **no landing act**: the board lands a piece privately, inside a tick or a drop, so a piece that comes to rest under gravity leaves its opening unclosed and the next piece's drop closes it — two placements matched at one closing entry, with the count right and the correlation wrong, and nothing to signal it. And **entry identifiers are not staging-invariant**: the three-node staging writes three seeding acts where one process writes one, shifting everything after, while the closing identifier is the only handle a match hands its reader. The trajectory is short — two placements — so this is an existence result about recognizability, not a measurement of it at scale.

Lab I — the domain grows. A score and a difficulty level were added to the board: the first computable by any client and therefore required to be single-valued, the second a rule over lines already recorded. The growth was additive in the strict sense. Over the whole domain directory the change is 98 lines added and 3 removed, across three files rather than the two new ones alone: `Scoring.cs` at 39 lines and `Difficulty.cs` at 38, plus 21 added lines in `Well.cs`. Of the 98 added, 30 are code; the other 68 are the domain's own documentation convention — 62 doc-comment lines and 6 blank — and a reader should not take them for filler or for logic. All 3 removed lines are doc-comment prose in `Well.cs`. No code was deleted, no signature changed and no verb removed, established by diffing a sorted inventory of the board's members rather than by inspection. The build graph came out byte-identical: no packages, no references, and the two new files declare no imports whatsoever. The domain's suite went from 44 tests to 60, the original 44 untouched.

The measurement this paper had never taken is the converse ripple. When the domain grew, all twelve host projects kept running with no edit — verified by running them before and after from a throwaway worktree at the pre-change commit, not merely by compiling them. The cost of *adopting* the new capability then fell only where it was wanted, and the twelve divide exactly: nothing for four of them, one line for five, four lines for each of the two browser hosts, and thirty-two for the input host, the only place where the level changes what happens. Those figures are the sum over both growth commits and both adoption commits; taking only the first two understates the input host, which acquires its thirty-two lines in the adoption commit rather than when the level was added. That last is where the boundary this paper predicted becomes visible: the domain owns what level the game is at, the staging owns how fast gravity ticks, and the domain still holds no clock — which a reflection test now enforces.

One finding is a caution rather than a result, and it applies to every replay measurement including this paper's. Three of the eight replay fixtures did not *fail*: they answered. A game recorded as having cleared twenty rows replayed as having cleared two; one recorded at six replayed at four; and the third, recorded at zero, replayed at zero — correct, because the acts it lost happened to clear nothing, so the truncation was invisible in exactly the scalars a reader

checks. The cause was a defect in the substrate's index rather than anything about the model, and it has since been fixed upstream, this lab being one of two independent reports that found it. The lesson survives the fix, and §8.4 states it as a rule: a replay checked only for internal consistency will pass while being wrong, so this lab now carries the state recorded at play time and asserts against it. That follows from what fusion means and is close to a tautology; no fused version was built. Until that exists, the comparison in this paragraph is an argument the paper makes, not a result it reports.

The nine laboratories' source and datasets accompany this paper as `paper09-data.zip`, laid out as the earlier papers in this series lay theirs out: one directory per lab under `labs/paper09-labX-…` holding its source and a README that states what it measures, what to run and which figure of this paper it produces, and a directory of the same name under `data/paper09-labX-…` holding its outputs and its write-up. A suite-level `labs/paper09-README.md` answers what the per-lab files cannot: that each lab stands alone and none depends on another, which one to run if only one is run, the order by cost, and the one interaction that can mislead — Lab A measures that the domain did *not* change while Labs G and I change it deliberately, so running Lab A's diff on their branches reports a difference that is a different measurement rather than a failed one. Two of the nine have no source of their own and say so in their README: Lab A, whose five stagings *are* hosts of the example and whose evidence is the empty diff across them, and Lab H, whose reaction is quoted in its write-up. Lab F is the largest, carrying the ported baseline in full. The labs are count-based, so no large raw sample log is produced or omitted, and every count cited above appears in full in the corresponding write-up.

Assembling that archive was itself a correction. The labs had been built on separate branches of the examples repository and never merged, so the material this section promises had been scattered across seven branches and reachable only by knowing which; a reader could not have obtained it from the repository named above. It is now consolidated, which is what makes the promise in the previous paragraph one a reader can act on.

One correction made during this work belongs in plain sight, because it touched the example, and an earlier draft of this appendix reported it as done when the artifact this paper deposits did not yet carry it. The frame each host pushes reaches its sink through a reaction over the journal, and a reaction over the domain observes recorded *operations*, never literal script text — a substrate rule, and a deliberate one. The example's actor had been emitting its verbs as literal text. So the push channel never engaged: no frame was written at all, which cost three labs their evidence rather than merely changing its form. The player hosts were unaffected, because each renders from its own query of the board; the *viewers* were the loss — the client-authored column-height view of §3 had no frame to read, the live viewer of Lab D had nothing to repaint, and each node of Lab B wrote no frame of its own.

The correction is now in the deposited artifact, and it is larger than the one

verb an earlier draft described. An operation is recorded as an operation only when it carries an argument, so the seed carries the board's dimensions and a spawn carries its piece; the five verbs that move a falling piece carry nothing that varies — there is no value in *move left* — and each therefore carries the name of the act it performs. The seam the actor speaks to no longer offers an unparametrized form at all, which makes the omission unrepresentable rather than merely discouraged, and the same conversion was applied to the two-role actor of Lab G. Two consequences are reported where they fall: entry counts move, because a template written once and a compact argument per act is a different encoding from one full sentence per call (§8.2), and Lab G's harness had been recovering the board's dimensions by reading them out of the seed's text, so it now reads them from the seed's arguments as it already did for a spawn.

The correction was to the example's actor, not to the domain: the domain's diff across the entire period, including this fix, is empty — which is the invariance this appendix reports, holding through a change to the machinery around it.

Code provenance

Source-code references in this paper are paths in **this repository**, and that is the one thing about provenance worth stating plainly: the example the labs measure is vendored here, at `labs/paper09-example/`, so the paper and the artifact it cites are one git history and there is a single commit to mark rather than two to reconcile. Example paths are cited relative to that directory — `domain/Well.cs:322`, `actor/IGameHost.cs:14` — and the ported baseline of Lab F, being a second and different arrangement, keeps its own prefix: `baseline-hex/…` under `labs/paper09-labF-ported-baseline/`. Line anchors resolve at the commit this paper is deposited from, and every one of them was re-resolved there before deposit rather than carried forward from the draft — nineteen in the example and the ported baseline, five in the framework.

One thing a vendored copy cannot carry is a history, and §8 rests on one: the dates that separate the domain's last commit from the stagings added twenty-four days later. That chronology is checkable only where it happened, in the repository the example was written in (`github.com/alvaroNCubo/puppeteer-examples`), on `main` at `ec9a2d4`: the domain's last commit is `fd8d94b`, the three additions of 23 July are `33f151c`, `8d7536e` and `974f62a`, and `git diff fd8d94b..main -- Tetris/domain/` is empty across the fifteen commits between. The copy here has one commit, the one that placed it, which is why Lab A of Appendix A is the single lab a reader cannot run against this repository and why its output is captured in `data/` for a reader who would rather not clone anything.

Framework source is cited in five files and nowhere else, and only where what is asserted is a property of the substrate that the example cannot establish: the scope of the emitting plane, which §8's constraint on admissible decompositions turns on (`Puppeteer/IOutputSink.cs`, `Puppeteer/StageHook.cs`); the

attachment of a destination to a live actor, which §9 uses to say when a destination is bound (`Choreography/Theater/PerformanceV2.cs`); and what a domain assembly must expose for the framework to read it, which Lab F's withdrawn concession turns on and which an example carrying an anchor cannot settle by itself (`Puppeteer/EventSourcing/DomainLibraries.cs`, `PuppeteerCli/AttachCommand.cs`). Those five anchors resolve against the **public** Puppeteer repository (`github.com/alvaroNCubo/puppeteer`) at commit `2673d57`, which is also the minimum the labs' engine requirement names. An earlier draft cited them against `dd67047`, a commit in the private Azure repository the runtime is developed in — a reference no reader could resolve, and the substantive provenance defect this pass fixed. Every other anchor in this paper is in the example, and every measurement reported is taken on the example rather than on framework internals. The example also contains two clients that no lab here counts — a webcam gesture input and an emulated constrained device — left out because their hardware legs were never run, so there is nothing measured to report about them.

One thing about the labs' provenance belongs here rather than beside each count, and it is more specific than a general disclaimer. This draft's measurements were taken while the example was still moving, and the labs do not all sit at one state of it: some were taken after the actor correction reported at the end of Appendix A and some before it. That correction changes how verbs are recorded, so it changes any figure computed from entry counts. One such figure existed — the record ratio of Lab G — and it has since been re-taken on the corrected example at the substrate commit this paper cites, which moved it and left everything else in that lab standing. No figure now reported comes from an uncorrected state. The correction never touched the figures the paper's argument rests on: a diff over a source directory, a count of project references, a count of hosts that needed editing, and a comparison of final board states are all indifferent to how an act was written into the journal. Every lab is nonetheless to be re-verified at the single published commit and reported at that commit as part of the deposit, and where a count is sensitive to the example's state the appendix says so rather than leaving a reader to reconcile dates.

A word on what is and is not public, since the two are easy to run together. The commit pinned above is in that public repository: `2673d57` is its `main`, so a reader who visits it today finds the source the five framework anchors resolve against. Archiving that snapshot under a Software Heritage persistent identifier — as the preceding paper in this series does — belongs to the deposit of this version, and this section will carry that identifier then. One cleanup was weighed for the same step and deliberately not made: the domain's assembly carries an `InternalsVisibleTo` grant to the console host which no host now exercises (Lab E reports it). Withdrawing it would rest the fence on the test suite's grant alone — tidier, and a change to what Lab E measured — so it stays as reported. The example's own anchors need no repository visit at all: they are in the accompanying archive, `paper09-data.zip`, whose contents are listed above. The three-machine lab reproduces from a single script, `labs/paper09-labB-three-`

`machines/docker/run-demo.sh`, against a local container runtime. The labs are count-based, so no large raw sample log is produced or omitted.

Acknowledgments

The author used large language models (including Claude and ChatGPT) as editorial assistants for language refinement, structural feedback, and literature navigation. All original ideas, terminology, theoretical constructs, and technical content presented in this work are solely the author's.

One disclosure belongs with the labs rather than with the writing, because a file it produced is cited above as evidence. The automated player of Lab C is an instance of a large language model, driving the game over a pipe; the view it reads the board through — the column-height projection of §3 (`tools/pile-scan.ps1`) — was written by that instance during the lab, with the author's permission, to suit the form in which it reasons. That the adapter was authored by the client rather than by the author of the domain is not incidental to the argument. It is the sharpest case available of a projection belonging to the party that reads it, and it is reported here so that a reader can weigh the evidence knowing how it came about.

References

Entries without a refereed venue — practitioner articles, talks, screencasts, and technical reports — are given with their access date and a permanent archival snapshot, and are marked as such where the argument leans on them; where such a source sits behind a paywall, a freely available presentation of the same idea is listed beside it.

The letter suffixes on this series' own entries denote the paper's number in the series, not the order of appearance here — `2026c` is Paper 3, `2026d` Paper 4, `2026g` Paper 7, `2026h` Paper 8 — so that a given paper carries the same label wherever in the series it is cited. Letters absent below belong to papers this one does not cite.

Agha, G. (1986). *Actors: A model of concurrent computation in distributed systems*. MIT Press.

Apel, S., Batory, D., Kästner, C., & Saake, G. (2013). *Feature-oriented software product lines: Concepts and implementation*. Springer. https://doi.org/10.1007/978-3-642-37521-7

Armstrong, J. (2003). *Making reliable distributed systems in the presence of software errors* [Doctoral dissertation, Royal Institute of Technology (KTH), Stockholm].

Bernhardt, G. (2012a). *Boundaries* [Conference talk, Software Craftsmanship North America; practitioner source, no refereed venue, freely available]. Destroy All Software. https://www.destroyallsoftware.com/talks/boundaries (accessed

26 July 2026; archived at https://web.archive.org/web/20260703082615/https://www.destroyallsoftware.com/talks/boundaries)

Bernhardt, G. (2012b). *Functional core, imperative shell* [Screencast; practitioner source, no refereed venue, behind a paywall — see 2012a for the freely available presentation of the same idea]. Destroy All Software. https://www.destroyallsoftware.com/screencasts/catalog/functional-core-imperative-shell (accessed 25 July 2026; archived at https://web.archive.org/web/20260723235945/https://www.destroyallsoftware.com/screencasts/catalog/functional-core-imperative-shell)

Brooks, F. P., Jr. (1987). No silver bullet: Essence and accidents of software engineering. *Computer*, 20(4), 10–19. https://doi.org/10.1109/MC.1987.1663532

Clements, P., & Northrop, L. (2001). *Software product lines: Practices and patterns*. Addison-Wesley.

Cockburn, A. (2005). *Hexagonal architecture* [Practitioner article, no refereed venue]. https://alistair.cockburn.us/hexagonal-architecture/ (accessed 25 July 2026; archived at https://web.archive.org/web/20260708163140/https://alistair.cockburn.us/hexagonal-architecture)

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387. https://doi.org/10.1145/362384.362685

Eugster, P. T., Felber, P. A., Guerraoui, R., & Kermarrec, A.-M. (2003). The many faces of publish/subscribe. *ACM Computing Surveys*, 35(2), 114–131. https://doi.org/10.1145/857076.857078

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Fowler, M. (2004). *Inversion of control containers and the dependency injection pattern* [Practitioner article, no refereed venue]. https://martinfowler.com/articles/injection.html (accessed 25 July 2026; archived at https://web.archive.org/web/20260722213553/https://martinfowler.com/articles/injection.html)

Fowler, M. (2005). *Event sourcing* [Practitioner article, no refereed venue]. https://martinfowler.com/eaaDev/EventSourcing.html (accessed 25 July 2026; archived at https://web.archive.org/web/20260714065500/https://martinfowler.com/eaaDev/EventSourcing.html)

Garlan, D., & Notkin, D. (1991). Formalizing design spaces: Implicit invocation mechanisms. In *VDM '91: Formal Software Development Methods*, Lecture Notes in Computer Science (Vol. 551, pp. 31–44). Springer. https://doi.org/10.1007/3-540-54834-3_5

Gregor, S. (2006). The nature of theory in information systems. *MIS Quarterly*, 30(3), 611–642.

Helland, P. (2005). Data on the outside versus data on the inside. *Proceedings of the 2nd Biennial Conference on Innovative Data Systems Research (CIDR '05)*, 144–153. (Reprinted in *Communications of the ACM*, 63(11), 111–118, 2020. https://doi.org/10.1145/3410623)

Helland, P. (2015). Immutability changes everything. *Proceedings of the 7th Biennial Conference on Innovative Data Systems Research (CIDR '15)*. (Also published in *ACM Queue*, 13(9), 2015, https://doi.org/10.1145/2857274.2884038, and in *Communications of the ACM*, 59(1), 64–70, 2016, https://doi.org/10.1145/2844112)

Hewitt, C., Bishop, P., & Steiger, R. (1973). A universal modular ACTOR formalism for artificial intelligence. *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI)*, 235–245.

Huang, P., Guo, C., Zhou, L., Lorch, J. R., Dang, Y., Chintalapati, M., & Yao, R. (2017). Gray failure: The Achilles' heel of cloud-scale systems. *Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS '17)*, Whistler, BC, Canada. https://doi.org/10.1145/3102980.3103005

Johnson, R. E., & Foote, B. (1988). Designing reusable classes. *Journal of Object-Oriented Programming*, 1(2), 22–35.

Kaldor, J., Mace, J., Bejda, M., Gao, E., Kuropatwa, W., O'Neill, J., Ong, K. W., Schaller, B., Shan, P., Viscomi, B., Venkataraman, V., Veeraraghavan, K., & Song, Y. J. (2017). Canopy: An end-to-end performance tracing and analysis system. *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP '17)*, 34–50. https://doi.org/10.1145/3132747.3132749

Kleppe, A., Warmer, J., & Bast, W. (2003). *MDA explained: The model driven architecture — Practice and promise*. Addison-Wesley.

Kleppmann, M., & Kreps, J. (2015). Kafka, Samza and the Unix philosophy of distributed data. *IEEE Data Engineering Bulletin*, 38(4), 4–14.

Kratzke, N., & Quint, P.-C. (2017). Understanding cloud-native applications after 10 years of cloud computing — A systematic mapping study. *Journal of Systems and Software*, 126, 1–16.

Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7), 558–565. https://doi.org/10.1145/359545.359563

Laurel, B. (1991). *Computers as theatre*. Addison-Wesley. (2nd ed., Addison-Wesley Professional, 2013.)

Mace, J., Roelke, R., & Fonseca, R. (2015). Pivot tracing: Dynamic causal monitoring for distributed systems. *Proceedings of the 25th Symposium on Operating Systems Principles (SOSP '15)*, 378–393. https://doi.org/10.1145/2815400.2815415

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Mellor, S. J., & Balcer, M. J. (2002). *Executable UML: A foundation for model-driven architecture*. Addison-Wesley.

Meyer, B. (1988). *Object-oriented software construction*. Prentice Hall.

Moseley, B., & Marks, P. (2006). *Out of the tar pit* [Presented at Software Practice Advancement (SPA 2006); no formal proceedings]. https://curtclifton.net/papers/MoseleyMarks06a.pdf (accessed 25 July 2026; archived at https://web.archive.org/web/20260718134750/https://curtclifton.net/papers/MoseleyMarks06a.pdf)

Murphy, G. C., Notkin, D., & Sullivan, K. (1995). Software reflexion models: Bridging the gap between source and high-level models. *Proceedings of the 3rd ACM SIGSOFT Symposium on the Foundations of Software Engineering (SIGSOFT '95)*; *ACM SIGSOFT Software Engineering Notes*, 20(4), 18–28. https://doi.org/10.1145/222132.222136

Murphy, G. C., Notkin, D., & Sullivan, K. J. (2001). Software reflexion models: Bridging the gap between design and implementation. *IEEE Transactions on Software Engineering*, 27(4), 364–380. (The extended journal version of the 1995 paper, and the canonical citable form.)

Overeem, M., Spoor, M., & Jansen, S. (2017). The dark side of event sourcing: Managing data conversion. *Proceedings of the 24th IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER '17)*, 193–204. https://doi.org/10.1109/SANER.2017.7884621

Overeem, M., Spoor, M., Jansen, S., & Brinkkemper, S. (2021). An empirical characterization of event sourced systems and their schema evolution — Lessons from industry. *Journal of Systems and Software*, 178, 110970.

Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12), 1053–1058. https://doi.org/10.1145/361598.361623

Passos, L., Terra, R., Valente, M. T., Diniz, R., & Mendonça, N. (2010). Static architecture-conformance checking: An illustrative overview. *IEEE Software*, 27(5), 82–89.

Perry, D. E., & Wolf, A. L. (1992). Foundations for the study of software architecture. *ACM SIGSOFT Software Engineering Notes*, 17(4), 40–52. https://doi.org/10.1145/141874.141884

Peyton Jones, S., & Wadler, P. (1993). Imperative functional programming. *Proceedings of the 20th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '93)*, 71–84. https://doi.org/10.1145/158511.158524

Rivera, A. (2026c). Reactions and the partition: opt-in eventual consistency in actor-native systems. *Puppeteer Papers Series*, Paper 3 [Preprint]. Zenodo. https://doi.org/10.5281/zenodo.20792156

Rivera, A. (2026d). Preserving semantic continuity across actors: a tell-based approach without orchestration. *Puppeteer Papers Series*, Paper 4 [Preprint]. Zenodo. https://doi.org/10.5281/zenodo.21207062

Rivera, A. (2026g). After the substrate: building software without a datacenter. *Puppeteer Papers Series*, Paper 7 [Preprint]. Zenodo. https://doi.org/10.5281/zenodo.20398998

Rivera, A. (2026h). Inference without Authority: the three authorities that govern an output. *Puppeteer Papers Series*, Paper 8 [Preprint]. Zenodo. https://doi.org/10.5281/zenodo.21499637

Sambasivan, R. R., Shafer, I., Mace, J., Sigelman, B. H., Fonseca, R., & Ganger, G. R. (2016). Principled workflow-centric tracing of distributed systems. *Proceedings of the 7th ACM Symposium on Cloud Computing (SoCC '16)*, 401–414. https://doi.org/10.1145/2987550.2987568

Shaw, M. (2003). Writing good software engineering research papers. *Proceedings of the 25th International Conference on Software Engineering (ICSE '03)*, 726–736.

Sigelman, B. H., Barroso, L. A., Burrows, M., Stephenson, P., Plakal, M., Beaver, D., Jaspan, S., & Shanbhag, C. (2010). *Dapper, a large-scale distributed systems tracing infrastructure* (Technical Report dapper-2010-1) [Technical report, no refereed venue]. Google Inc. https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/ (accessed 25 July 2026; archived at https://web.archive.org/web/20260705194849/https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/)

Stol, K.-J., & Fitzgerald, B. (2018). The ABC of software engineering research. *ACM Transactions on Software Engineering and Methodology*, 27(3), Article 11. https://doi.org/10.1145/3241743

Sullivan, K. J., & Notkin, D. (1992). Reconciling environment integration and software evolution. *ACM Transactions on Software Engineering and Methodology*, 1(3), 229–268.

Terra, R., & Valente, M. T. (2009). A dependency constraint language to manage object-oriented software architectures. *Software: Practice and Experience*, 39(12), 1073–1094. https://doi.org/10.1002/spe.931

Terry, D. B., Demers, A. J., Petersen, K., Spreitzer, M. J., Theimer, M. M., & Welch, B. B. (1994). Session guarantees for weakly consistent replicated data. *Proceedings of the Third International Conference on Parallel and Distributed Information Systems (PDIS '94)*, 140–149.

Waldo, J., Wyant, G., Wollrath, A., & Kendall, S. (1994). *A note on distributed*

computing (Technical Report SMLI TR-94-29). Sun Microsystems Laboratories. (Reprinted in J. Vitek & C. Tschudin, Eds., *Mobile object systems: Towards the programmable internet*, Lecture Notes in Computer Science, Vol. 1222, pp. 49–64. Springer, 1997. https://doi.org/10.1007/3-540-62852-5_6)

Wiggins, A. (2011). *The twelve-factor app* [Practitioner source, no refereed venue]. https://12factor.net (accessed 25 July 2026; archived at https://web.archive.org/web/20260724222316/https://12factor.net/)

Young, G. (2010). *CQRS documents* [Practitioner source, no refereed venue]. https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf (accessed 25 July 2026; archived at https://web.archive.org/web/20240317190715/https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf)